@ P. Siagian

Internet Control Message Protocol version 4 (ICMPv4)

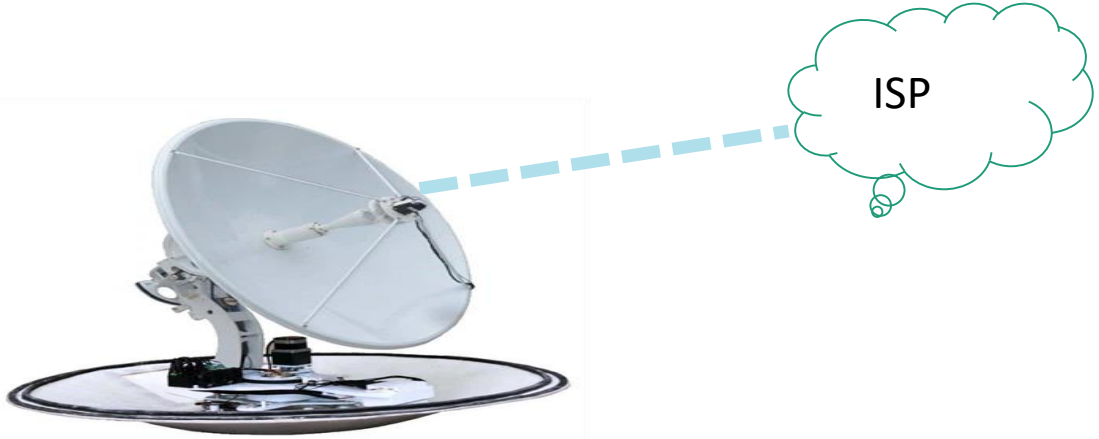
https://github.com/siagianp

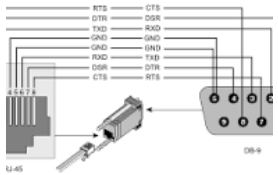
https://www.youtube.com/@AmelOline/video

https://github.com/amelcharolinesgn2/IoT_simulator-mqtt-NodeRed

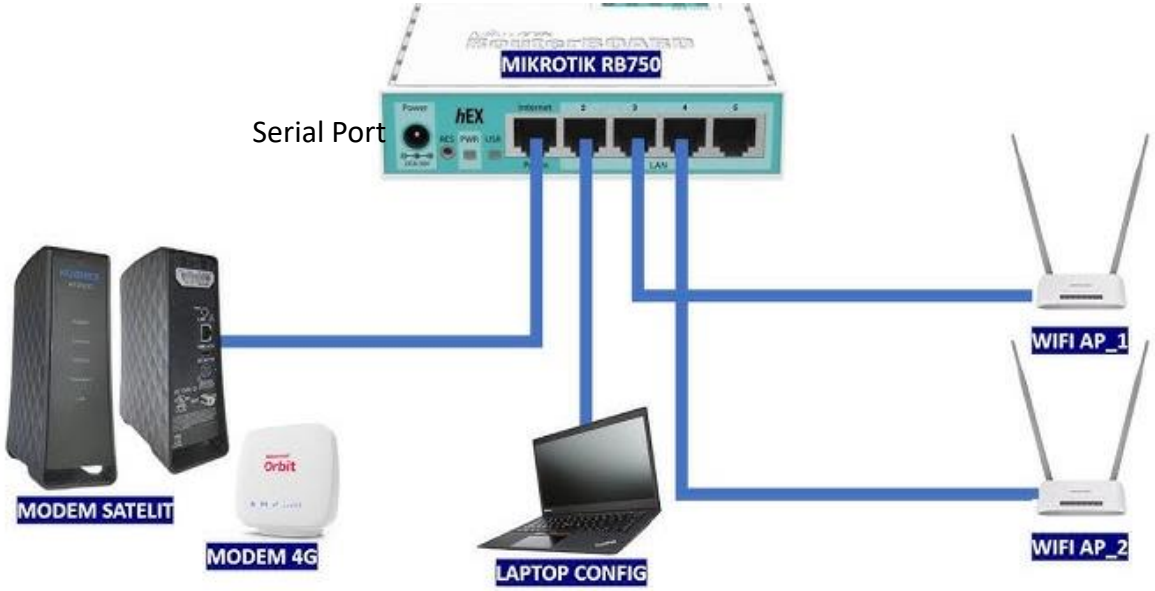
github.com/amelcharolinesgn2/ANJAR

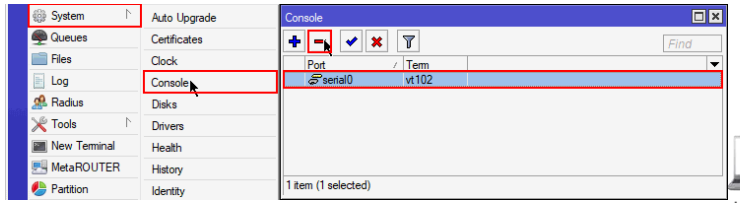
Mikrotik

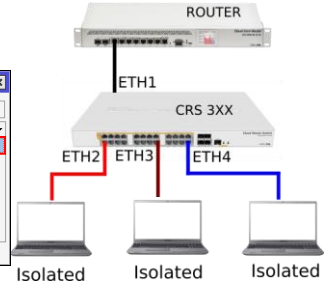




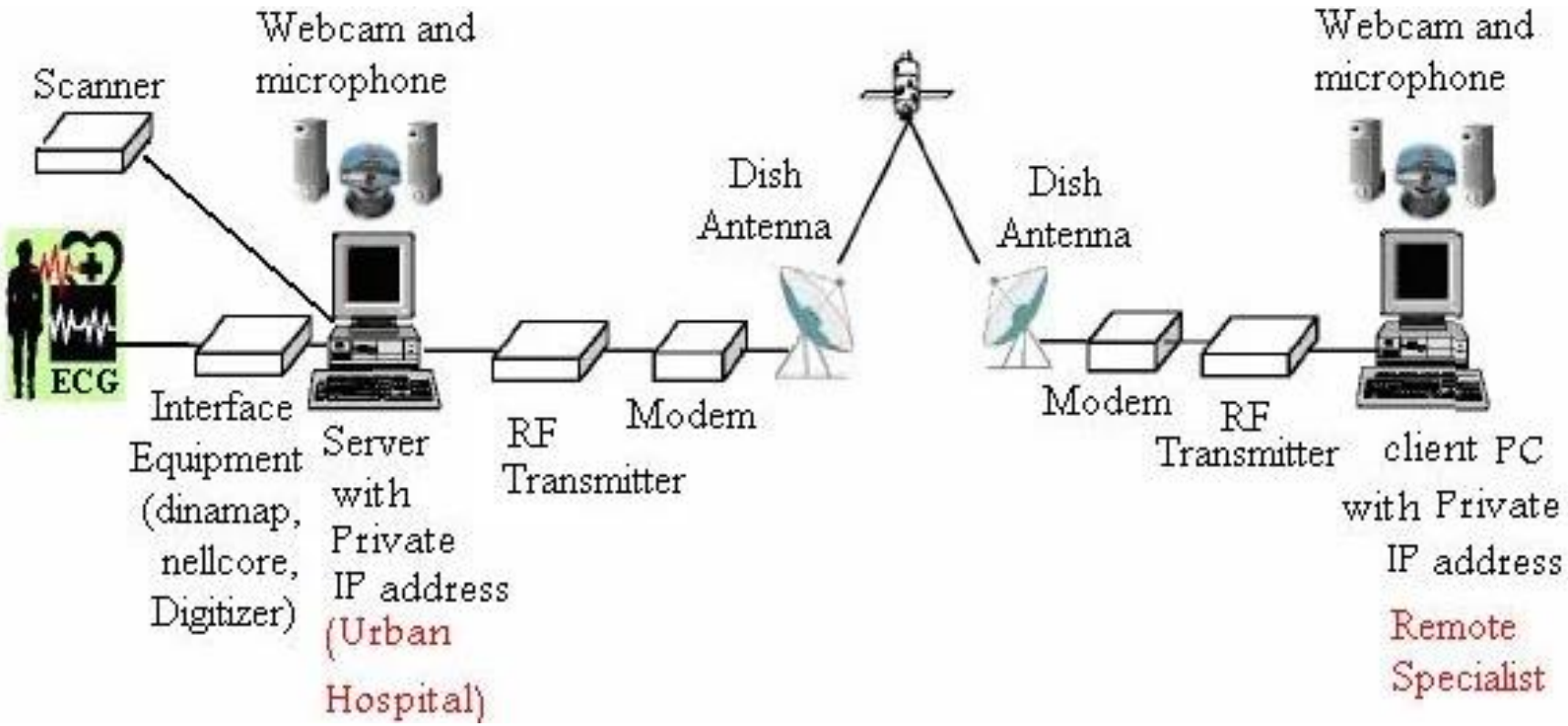
serial port mikrotik to vsat



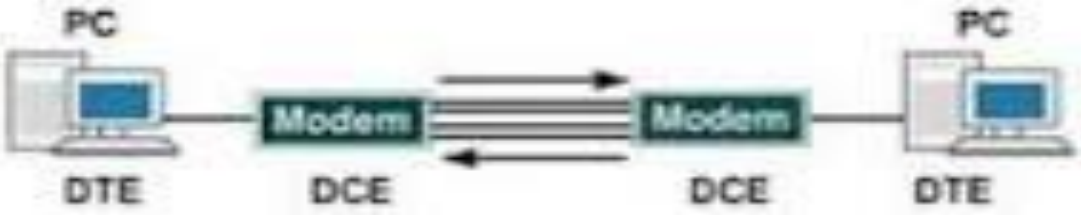


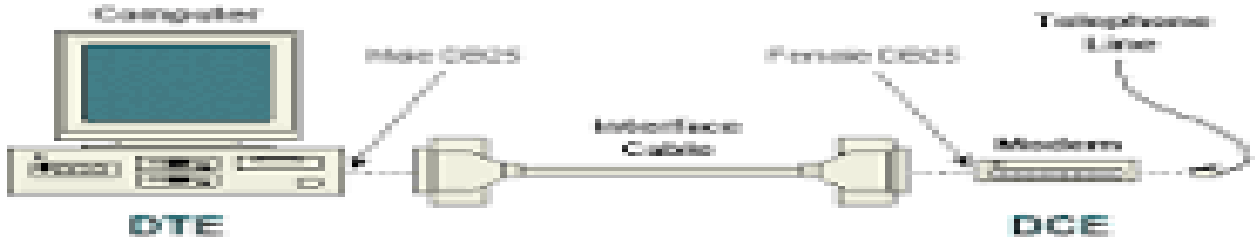


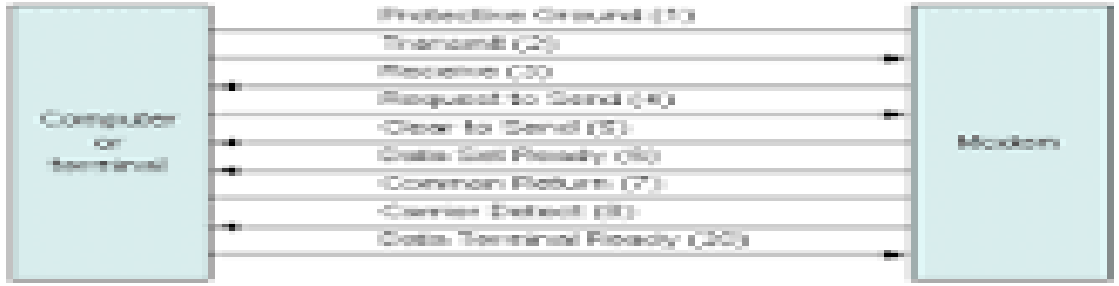
Modem

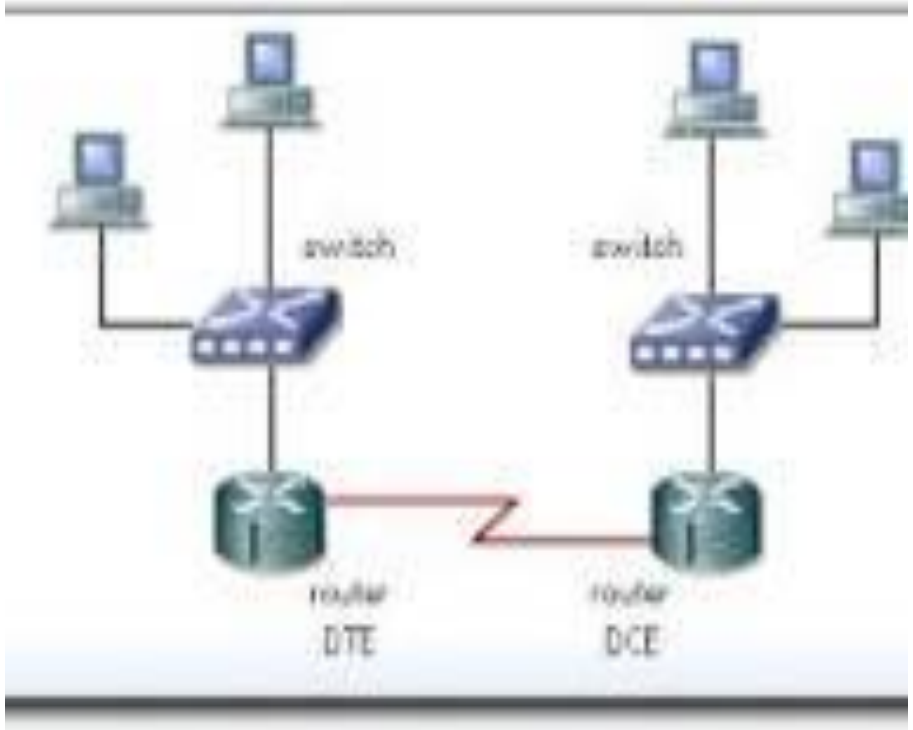

Webcam and microphone
Scanner
Webcam and microphone
Dish Antenna
Dish Antenna
ECG
Interface Equipment (dinamap, nellcore, Digitizer)
Server with Private IP address (Urban Hospital)
RF Transmitter
Modem
Modem
RF Transmitter
client PC with Private IP address
Remote Specialist

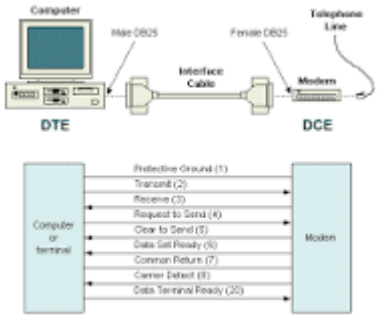
PC
PC
Modem
Modem
DTE
DCE
DCE
DTE

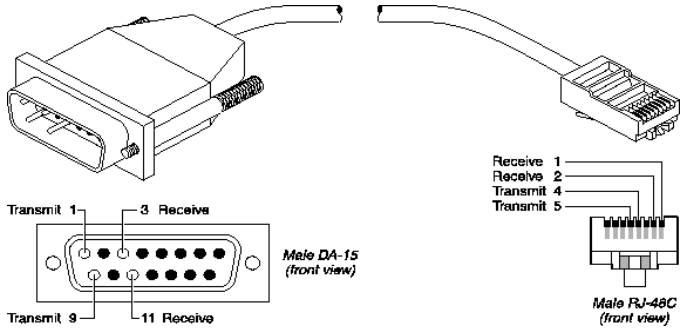

PC
PC
Modem
Modem
DTE
DCE
DCE
DTE

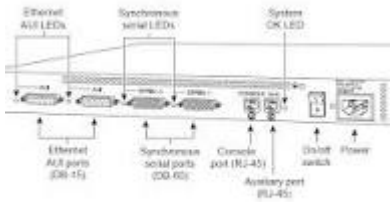

Computer
Male DB25
Female DB25
Telephone Line
Interface Cable
Modem
DTE
DCE

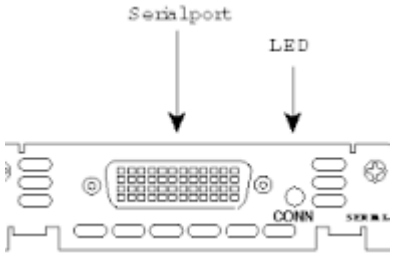

Protective Ground (1)
Transmit (2)
Receive (3)
Request to Send (4)
Clear to Send (5)
Data Set Ready (6)
Common Return (7)
Carrier Detect (8)
Data Terminal Ready (20)
Computer or terminal
Modem

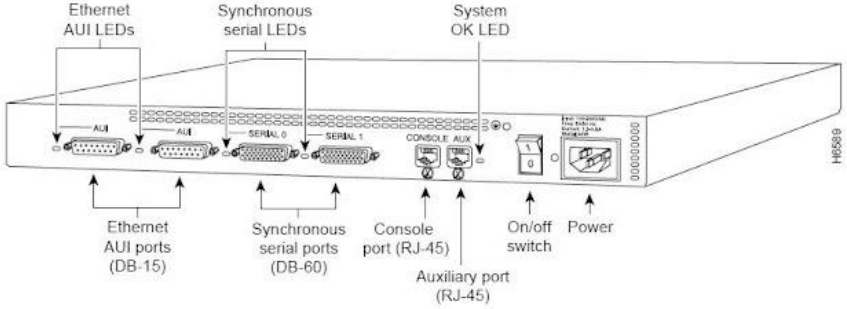

switch
switch
router
DTE
router
DCE

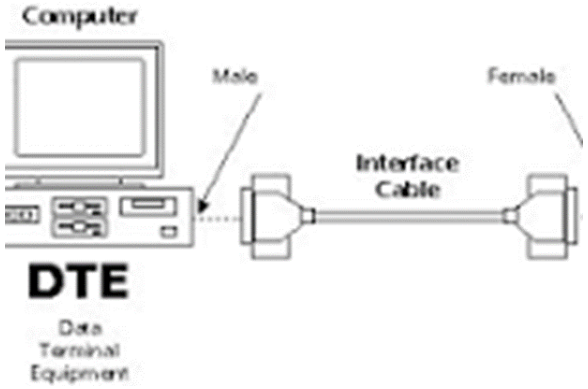

Computer
Male DB25
Female DB25
Telephone Line
Interface Cable
Modem
DTE
DCE
Protective Ground (1)
Transmit (2)
Receive (3)
Request to Send (4)
Clear to Send (5)
Data Set Ready (6)
Common Return (7)
Carrier Detect (8)
Data Terminal Ready (20)
Computer or terminal
Modem

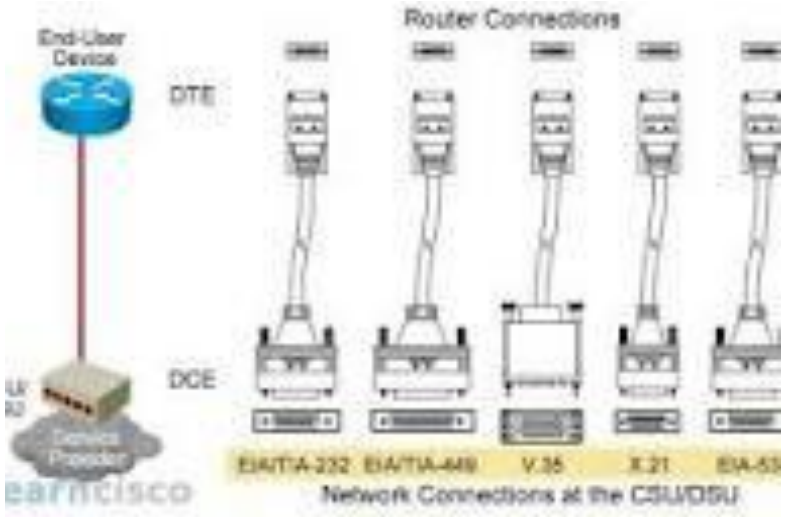

Transmit 1
3 Receive
Male DA-15 (front view)
Transmit 9
11 Receive
Receive 1
Receive 2
Transmit 4
Transmit 5
Male RJ-48C (front view)

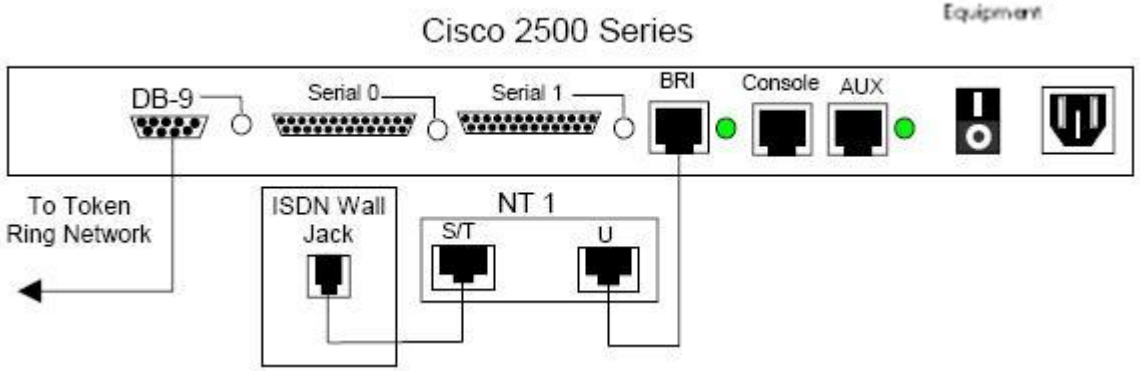

Ethernet AUI LEDs
Synchronous serial LEDs
System OK LED
Ethernet AUI ports (DB-15)
Synchronous serial ports (DB-60)
Console port (RJ-45)
Auxiliary port (RJ-45)
On/off switch
Power

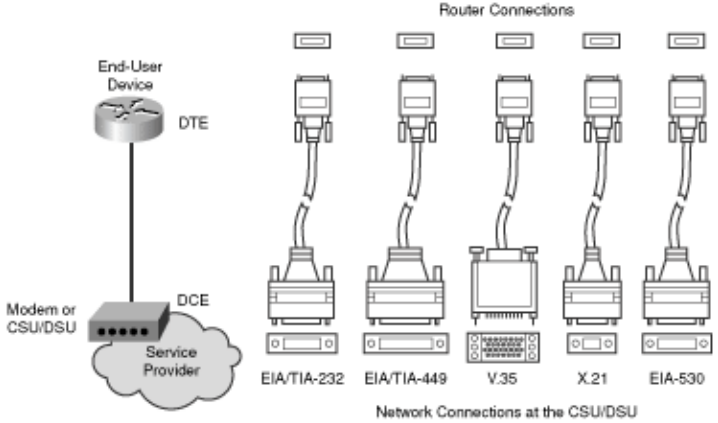

Serialport
LED
CONN

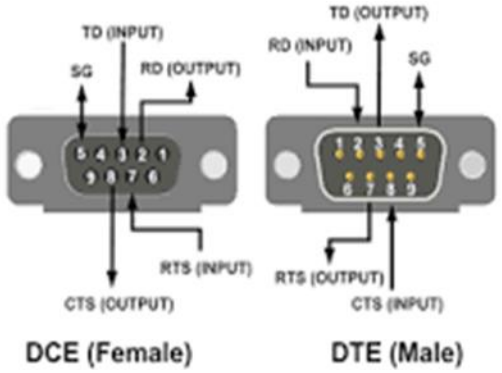

Ethernet AUI LEDs
Synchronous serial LEDs
System OK LED
AUI
AUI
SERIAL 0
SERIAL 1
CONSOLE AUX
Ethernet AUI ports (DB-15)
Synchronous serial ports (DB-60)
Console port (RJ-45)
Auxiliary port (RJ-45)
On/off switch
Power

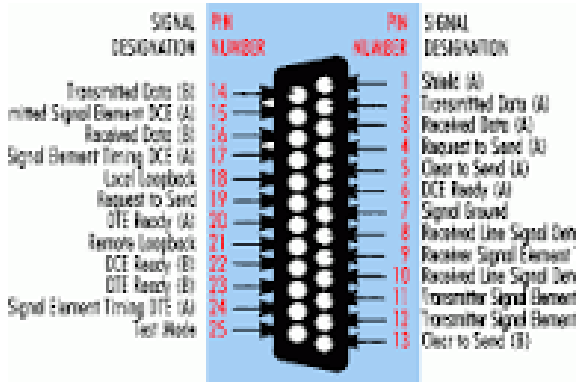

Computer
Male
Female
Interface Cable
DTE
Data Terminal Equipment


Router Connections
End-User Device
DTE
DCE
Service Provider
EIA/TIA-232
EIA/TIA-449
V.35
X.21
Network Connections at the CSU/DSU


Cisco 2500 Series
DB-9
Serial 0
Serial 1
BRI
Console
AUX
To Token Ring Network
ISDN Wall Jack
NT 1
S/T
U


Router Connections
End-User Device
DTE
DCE
Modem or CSU/DSU
Service Provider
EIA/TIA-232
EIA/TIA-449
V.35
X.21
EIA-530
Network Connections at the CSU/DSU



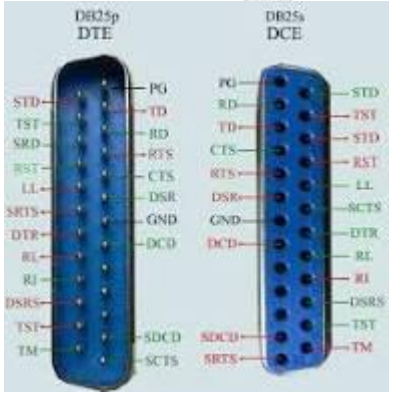


o Modem - Crossover Cable DB25 to DB25

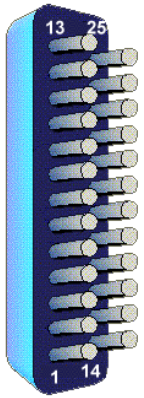
DCE to DCE

vice (Modem)	DB25	Connections	DCE Device (Modem)
RS-232 Signal Names		Signal Direction	Pin# DB25 RS-232 Sign
f Frame Ground	FG		#1 Shield to Frame Ground
Data (Tx)	TD		#2 Transmit Data (Tx)
Data (Rx)	RD		#3 Receive Data (Rx)
to Send	RTS		#4 Request to Send
Send	CTS		#5 Clear to Send
Ready	DSR		#6 Data Set Ready
ound/common (SG)	GND		#7 Signal Ground/common
Detector (DCD)	CD		#8 Carrier Detector (DCD)
minal Ready	DTR		#20 Data Terminal Ready
ator	RI		#22 Ring Indicator

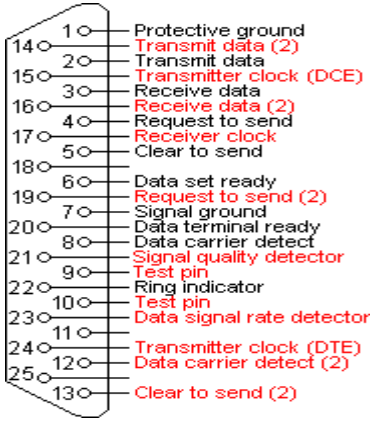
ent" cable for DTE to DTE also connects pins #6 & #8 together on each side sin
ns are reversed when devices are DTE to DTE.



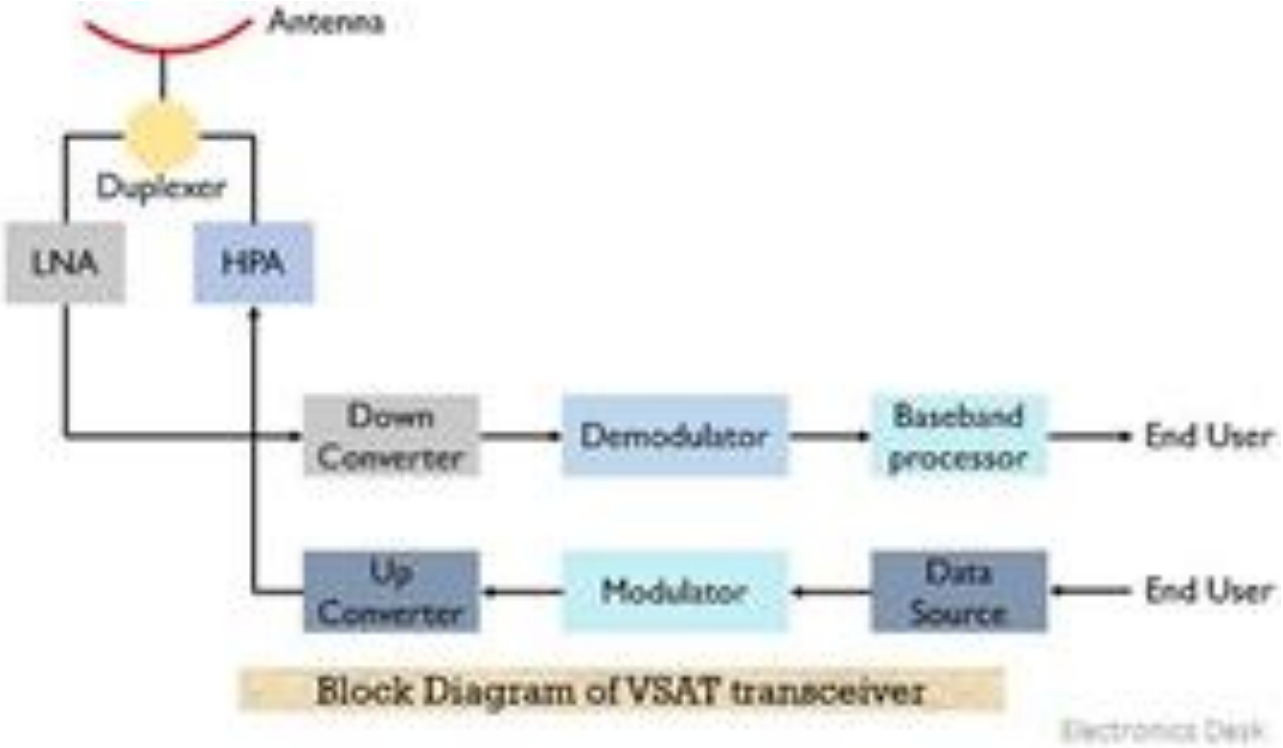
RS232 Pinout on DB25

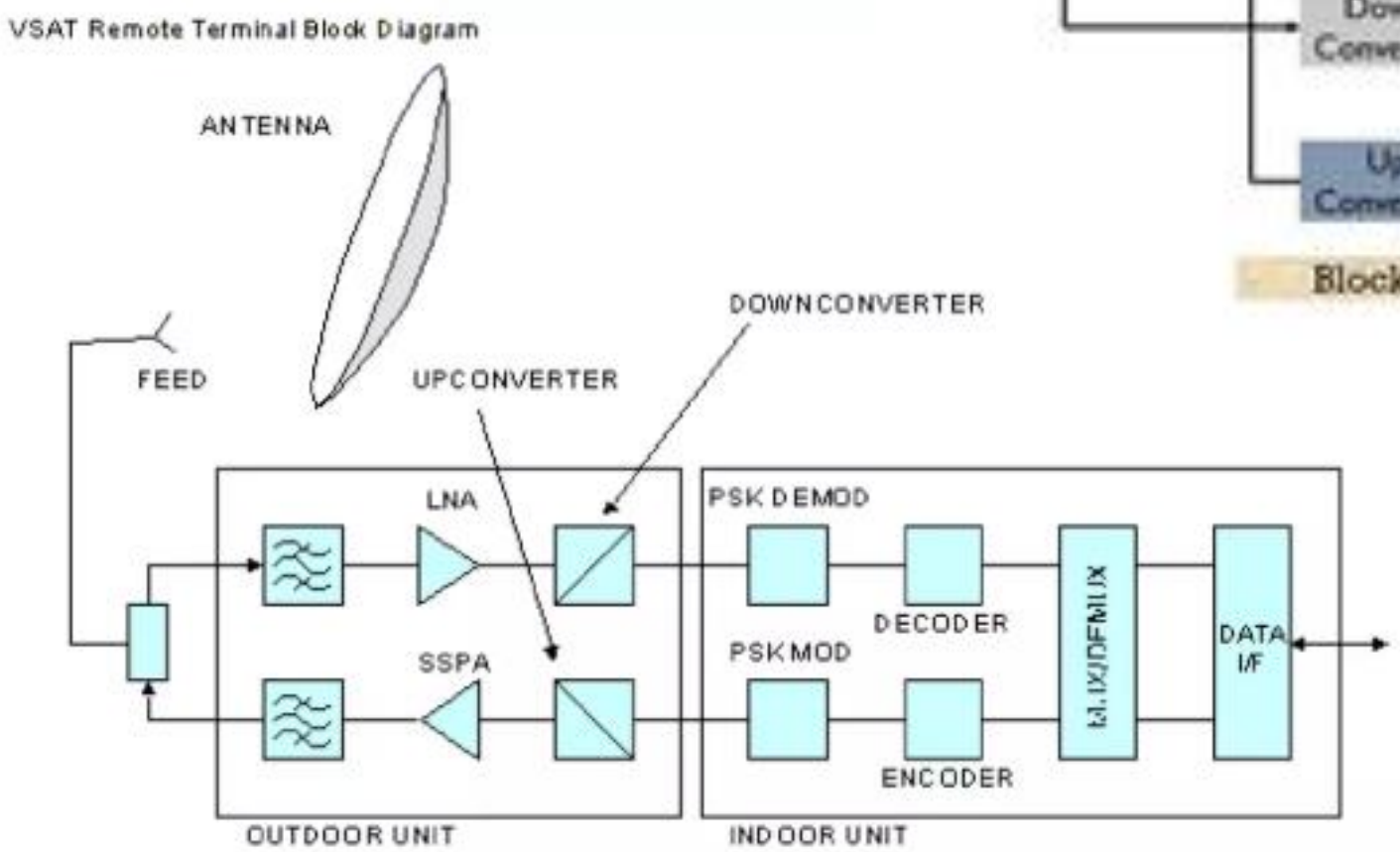


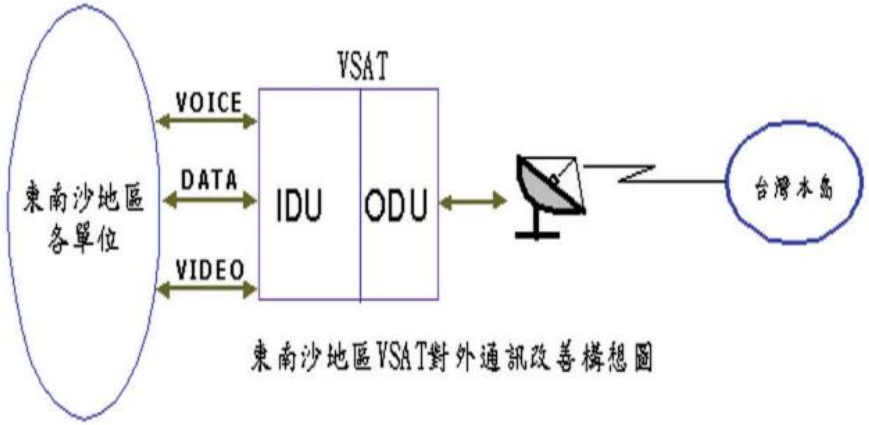
2 Transmit Data (TxD)
3 Receive Data (RxD)
4 Request to Send (RTS)
5 Clear to Send (CTS)
6 Dataset ready (DSR)
7 Signal Ground
8 Data Carrier Detect (DCD)
15 Transmit Clock
17 Receive Clock
20 Data Terminal Ready (DTR)
24 Auxiliary Clock



Satelit VSat

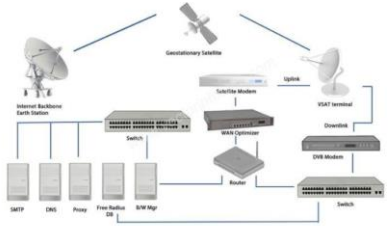


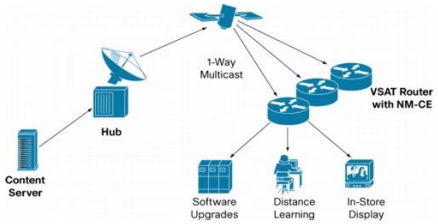




東南沙地區VSAT對外通訊改善構想圖

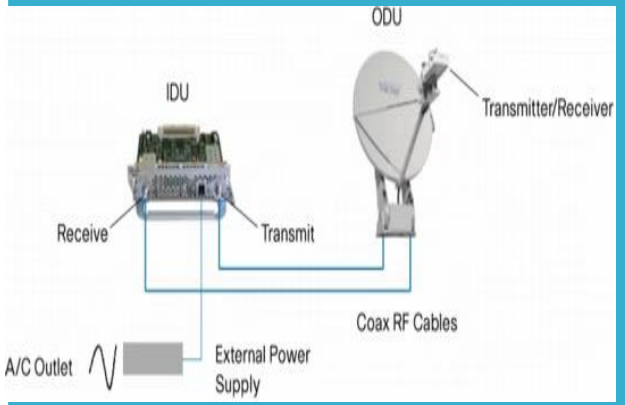
Satelit



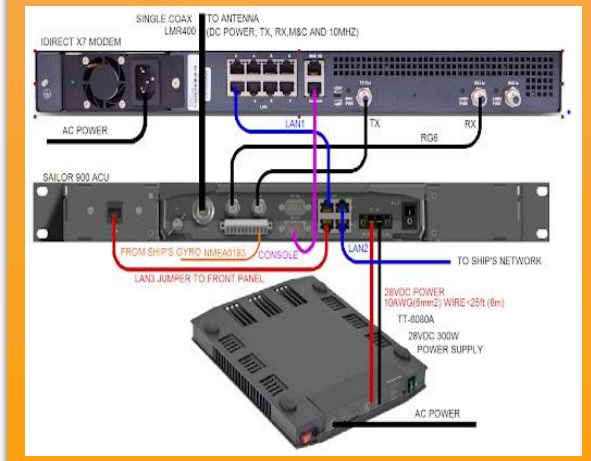


IP vsat

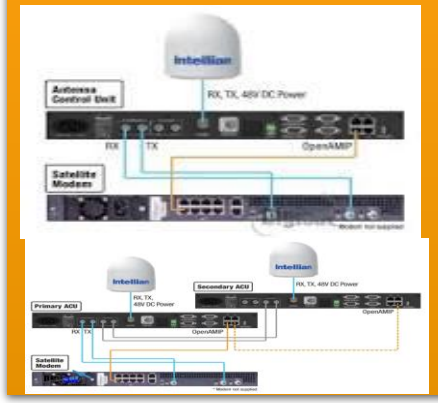
Cisco IP VSAT Satellite WAN Network Module for Cisco Integrated Services Routers



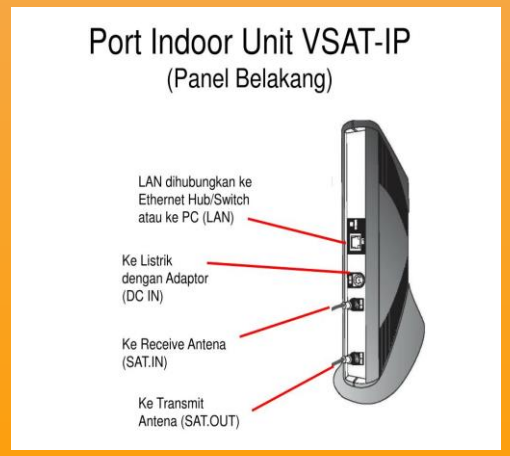
Sailor 900 VSAT

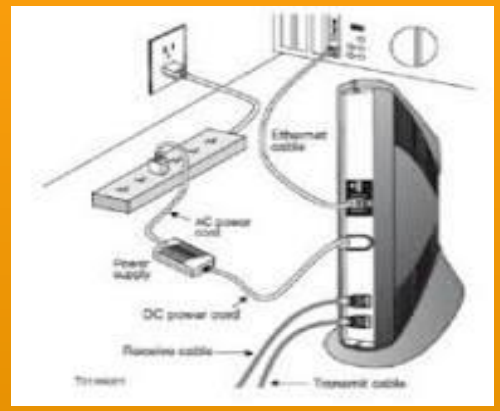


Intellian V60NX Ka Multi-Orbit Tracking VSAT Antenna



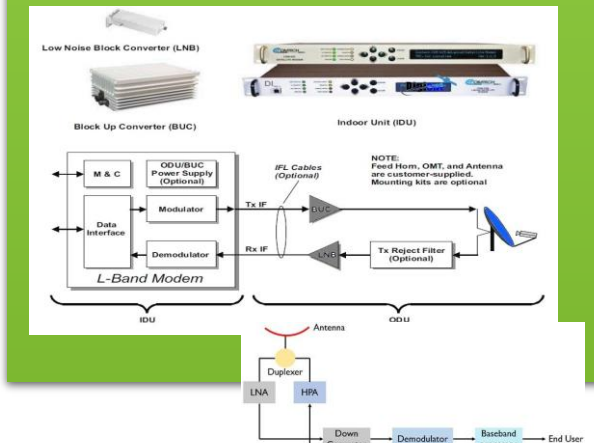
Port Indoor Unit VSAT-IP





Comtech EF Data CTOG-250 Advanced VSAT

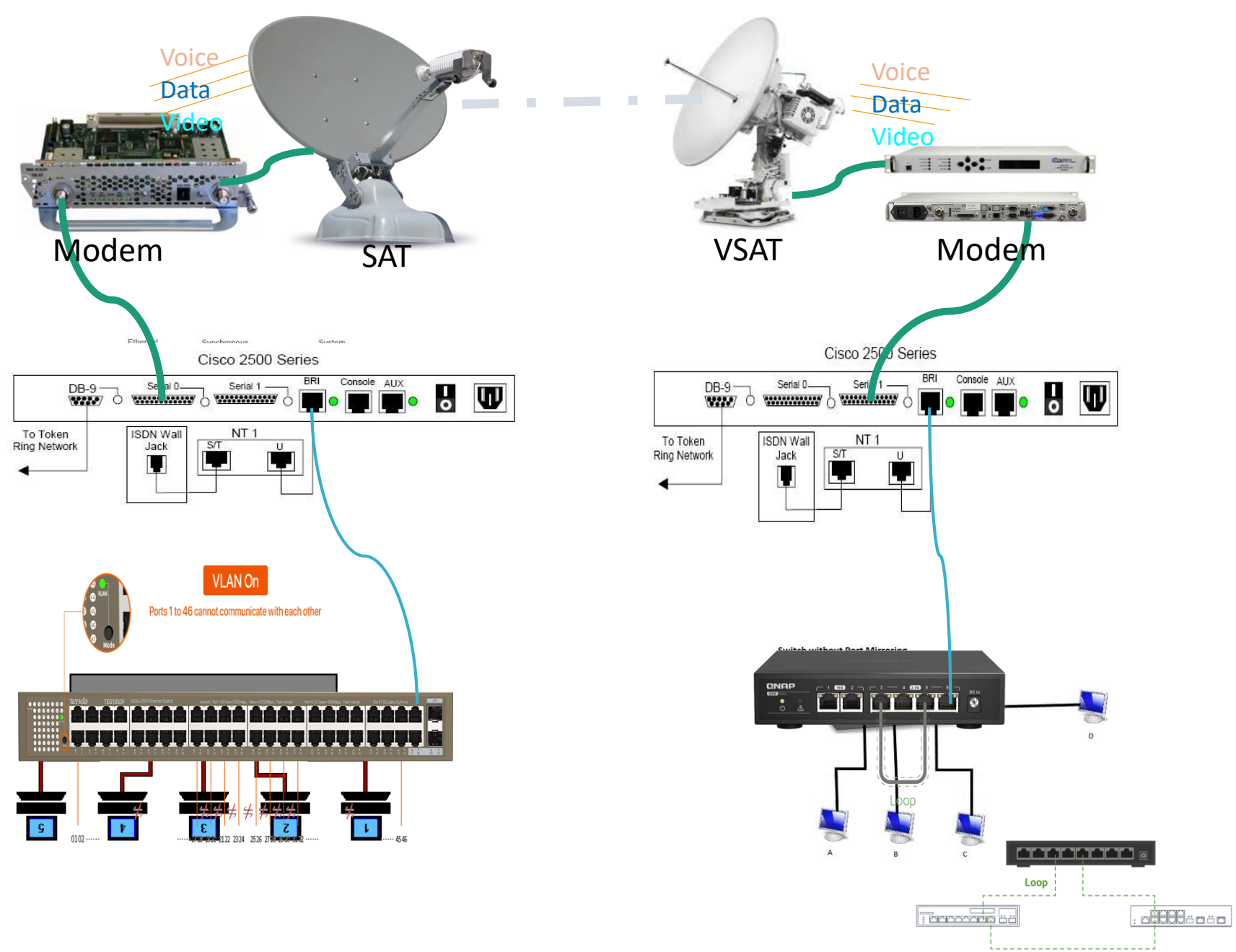
Comtech EF Data DST-C C-Band Single Channel

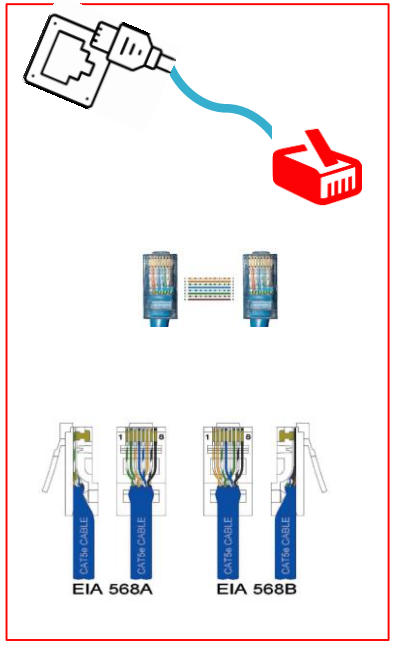


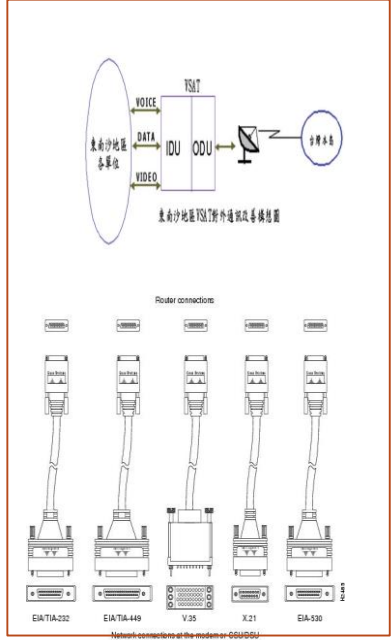
Comtech CDM-570L & CDM-570L

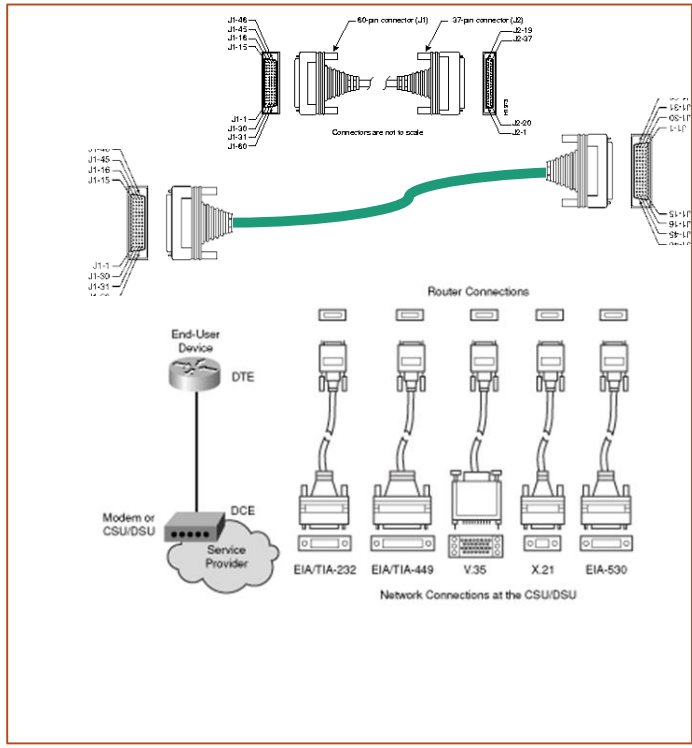
CDM-570/L, IP & EN Software

Comtech EF Data CDM-600


Voice
Data
Video
Modem
SAT
VSAT
Voice
Data
Video
Modem
Cisco 2500 Series
DB-9
Serial 0
Serial 1
BRI
Console
AUX
To Token Ring Network
ISDN Wall Jack
NT 1
S/T
U
Cisco 2500 Series
DB-9
Serial 0
Serial 1
BRI
Console
AUX
To Token Ring Network
ISDN Wall Jack
NT 1
S/T
U
VLAN On
Ports 1 to 46 cannot communicate with each other
Mode
Switch without Port Mirroring
QNAP
Loop
A
B
C
D
Loop

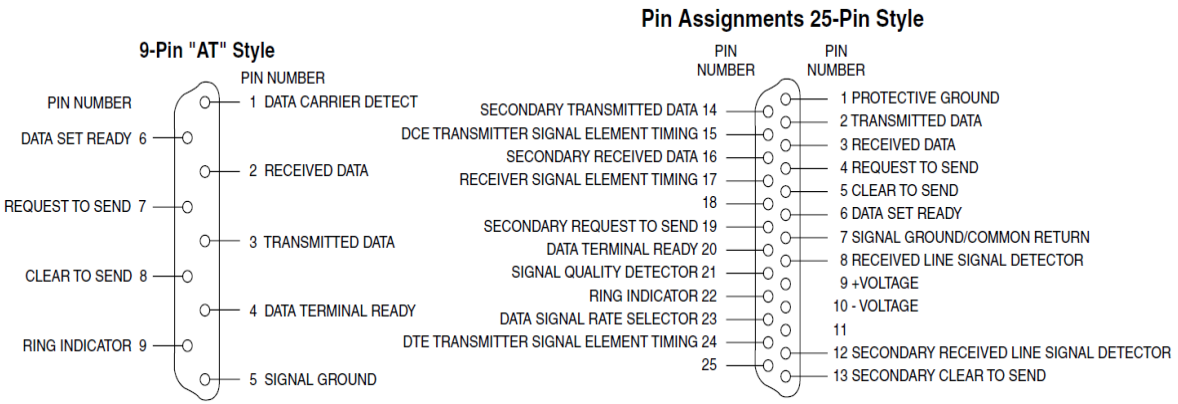

EIA 568A
EIA 568B


VSAT
VOICE
DATA
VIDEO
IDU
ODU
Router connections
EIA/TIA-232
EIA/TIA-449
V.35
X.21
EIA-530


60-pin connector (J1)
37-pin connector (J2)
Connectors are not to scale
Router Connections
End-User Device
DTE
Modem or CSU/DSU
DCE
Service Provider
EIA/TIA-232
EIA/TIA-449
V.35
X.21
EIA-530
Network Connections at the CSU/DSU

1. EIA-232

Komunikasi data EIA-232 pada awalnya dikembangkan untuk komunikasi data antara DTE (Data Terminal Equipment) dan DCE (Data Circuit Terminating Equipment). Pertukaran data dilakukan secara serial dengan cara mengirimkan data biner per-bit. EIA-232 pada awalnya bernama RS-232 (Recomended Standard 232). EIA-232 hanya dapat digunakan untuk menghubungkan dua perangkat untuk saling bertukar data atau yang biasa disebut dengan komunikasi point to point. Komunikasi data ini awalnya digunakan untuk menghubungkan antara komputer dan modem (modulator demodulator). Namun saat ini EIA-232 banyak digunakan sebagai standar komunikasi data pada perangkat industrial seperti PLC, HMI, Barcode Scanner dan lain-lain. Koneksi EIA-232 menggunakan konektor standar DB9 atau DB25 dengan fungsi masing-masing pin seperti pada Gambar 1. Perangkat DTE menggunakan konektor tipe male sedangkan perangkat DCE menggunakan konektor tipe female. Terdapat dua pin utama yang digunakan untuk bertukar data yaitu pin transmitted data dan pin received data. Pin transmitted data digunakan untuk mengirimkan data ke perangkat lain. Sedangkan pin received data digunakan untuk menerima data dari perangkat lain.



TIA/EIA

EIA (Electronic Industries Alliance)

1.2 Frame Data EIA-232

Frame data dapat juga disebut dengan format data. Standar EIA-232 mengatur format data sedemikian rupa agar terstandarisasi. Pada saat pengiriman data terjadi, sinyal yang dikirimnya tidak hanya sinyal data saja, namun ada tambahan sinyal lain yang dikirimkan bersamaan dengan satu paket data. Sinyal lain ini disertakan dengan maksud dan tujuan tertentu. Kumpulan sinyal data dan sinyal-sinyal tambahan ini disebut dengan frame data.

Frame data pada EIA-232 terdiri dari :

Start Bit, berukuran 1 bit dengan nilai selalui 0 yang berfungsi sebagai petanda awal pengiriman data
Data Bit, berukuran 5, 6, 7 atau 8 bit merupakan data yang akan ditransmisikan
Parity, berukuran 1 bit yang dapat digunakan untuk pengecekan error pada pertukaran data
Stop Bit, berukuran 1 atau 2 bit dengan nilai selalu 1 sebagai petanda akhir pengiriman data

Pada gambar di atas dapat dilihat bahwa satu frame data selal terdiri dari start bit, data bit, parity dan stop bit. Syarat ir multak jika ingin menggunakan standar EIA-232. Namun untu sinyal parity bisa digunakan atau tidak, sifatnya optional. Pad gambar di atas dapat dilihat bahwa pada saat idle atau tida ada data yang ditransmisikan, kondisi sinyal akan selalu high Data ditransmisikan diawali dengan mengirim start bit yan nilainya selalu low. Sinyal start bit ini bertujuan untu memberikan petanda bahwa transmisi data akan seger dilakukan. Setelah start bit ditransmisikan, kemudian diiku oleh 8 bit data. Besarnya data bit yang ditransmisikan jug tergantung dengan yang diinginkan. Bisa dipilih 5, 6, 7 atau bit data dalam 1 frame data. Pada umumnya data bit yan digunakan adalah 8 bit. Setelah 8 bit data ditransmisika kemudian diikuti oleh parity bit. Parity bit berguna sebaga sinyal untuk pengecekan error. Jika parity bit digunakan, And dapat memilih dua mode yaitu ODD atau EVEN. Pengeceka error menggunakan parity bit sangat sederhana, yaitu denga menghitung banyaknya sinyal high pada data bit. Metod pengecekan error tersebut lebih jelasnya seperti berikut

ODD (ganjil), jika jumlah sinyal high pada data bit adalah ganj maka bit parity akan bernilai 0 (low). Jika jumlah sinyal hig pada data bit adalah genap maka bit parity akan bernilai

Tabel Pengecekan Error Pada Bit Parity

Data Bit	Jumlah Bit 1 (high)	Even	Odd
00000000	0	0	1
01010001	3	1	0
01101001	4	0	1
01111111	7	1	0

Sinyal terakhir yang dikirimkan pada satu frame data adalah stop bit. Sinyal stop bit berfungsi untuk menandakan bahwa pengiriman data sudah berakhir. Sinyal stop bit dapat menggunakan 1 bit sinyal atau 2 bit sinyal, tergantung konfigurasi yang digunakan. Perlu digarisbawahi bahwa format frame data antara dua perangkat yang saling berkomunikasi harus sama. Jika tidak sama kemungkinan besar proses komunikasi data akan error.

2. EIA-485/EIA422

Cisco CAB-HD4-232MT High Density 4-port EIA-232 DTE Cable

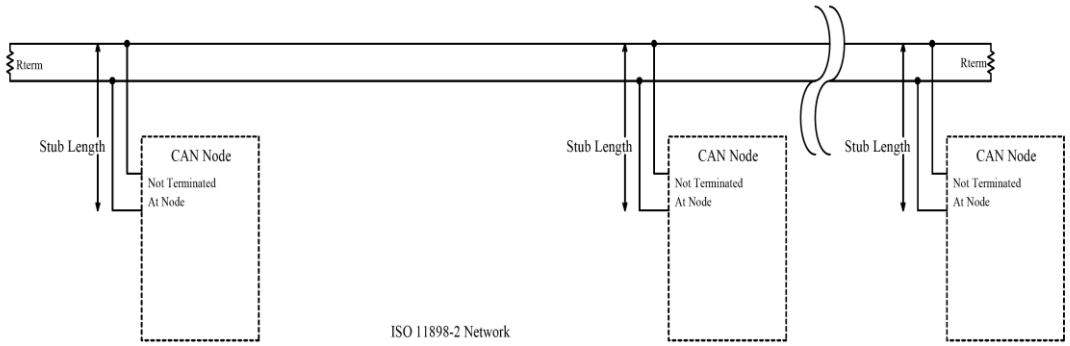
3.2 Frame Data CAN Bus

Karakteristik sinyal pada CAN Bus lebih mirip dengan sinyal pada EIA-485. Dimana terdapat dua buat terminal kabel yang digunakan untuk melakukan transmisi data. Perbedaan tegangan dari kedua terminal kabel ini yang digunakan untuk merepresentasikan sinyal high dan low dalam transmisi data. Kedua terminal pada CAN bus tersebut dikenal dengan nama CAN High dan CAN Low. Pada CAN Bus dikenal dua jenis sinyal yaitu dominant dan recessive. Dominant sama dengan sinyal 0 (low) sedangkan recessive sama dengan sinyal 1 (high).

Terdapat dua standar sinyal pada CAN Bus yaitu standar ISO 11898-2 atau lebih dikenal dengan nama High Speed CAN Bus. Standar kedua yaitu ISO 11898-3 atau lebih dikenal dengan nama Low Speed CAN Bus. Perbedaan dari kedua standar tersebut akan dijelaskan di bawah ini.

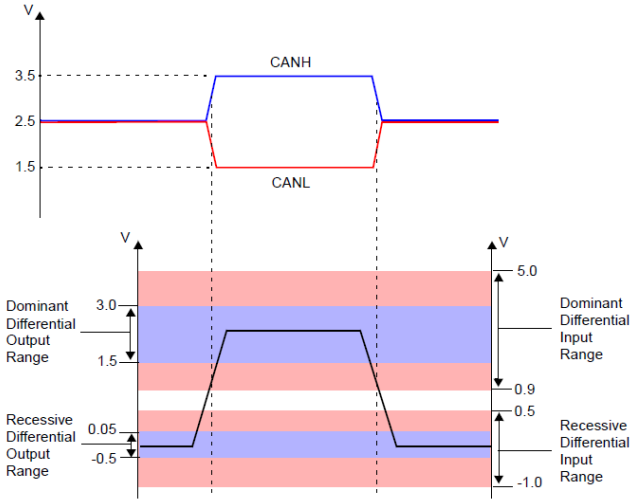
High Speed CAN Bus

High Speed CAN Bus menggunakan topologi jaringan model linear bus. Dimana semua node akan terkoneksi dalam sebuah jaringan yang memiliki dua buah titik akhir (endpoint). Semua node akan dihubungkan ke terminal CAN High dan CAN Low. Pada masing-masing endpoint akan dipasang resistor termination yang nilainya sebesar 120 ohm.



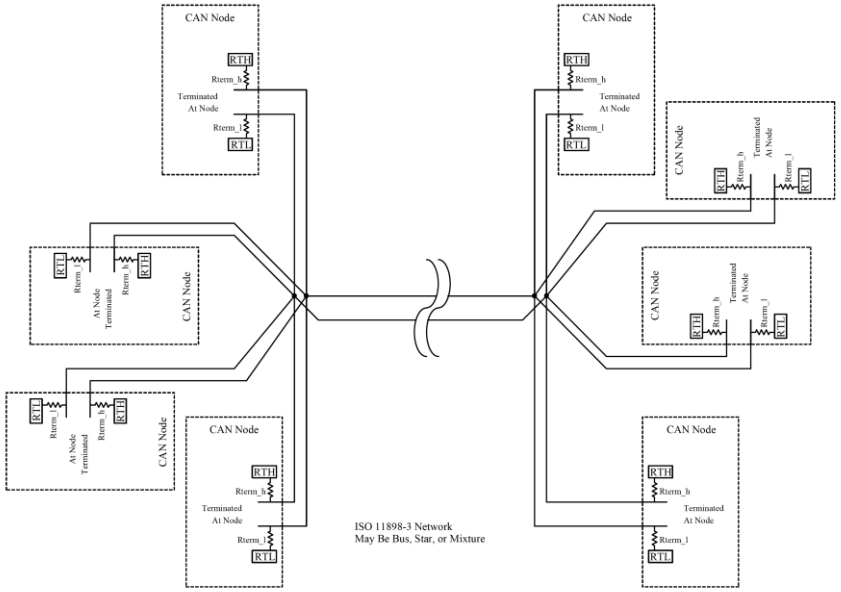
High Speed CAN Bus dapat beroperasi dengan kecepatan antara 40Kbps - 1Mbps. Karakteristik sinyal pada high speed CAN bus dapat dilihat di bawah ini :

- Recessive (high / 1) terjadi jika perbedaan tegangan pada terminal CAN High dan CAN Low antara -0.1 volt sampai +0.5 volt
- Dominant (low / 0) terjadi jika perbedaan tegangan pada terminal CAN High dan CAN Low antara +0.9 volt sampai +5 volt



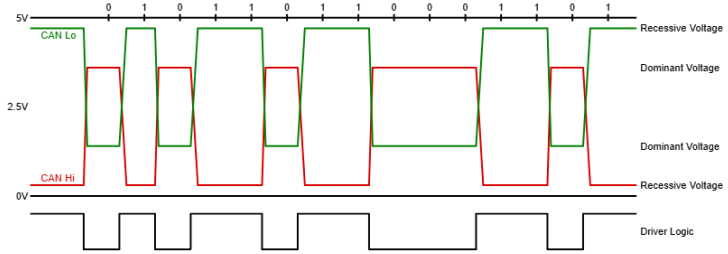
Low Speed CAN Bus

Low Speed CAN Bus menggunakan topologi jaringan linear bus, star bus atau gabungan dari star bus dan linear bus. Pada masing-masing node digunakan termination resistor sebesar 100 ohm pada terminal CAN High dan CAN Low. Termination resistor terpasang pada masing-masing node, tidak seperti pada high speed CAN bus yang hanya terpasang pada endpoint saja. Lebih jelasnya dapat dilihat pada topologi jaringan gabungan dari star dan linear bus.



Low Speed CAN Bus dapat beroperasi pada kecepatan 10Kbps - 125Kbps. Namun walaupun beroperasi di kecepatan yang lebih rendah, Low Speed CAN Bus lebih handal dalam penanganan error. Ini dikarenakan dengan kecepatan rendah, error yang terjadi akhibat noise atau penyebab yang lainnya lebih rendah. Low Speed CAN Bus biasanya disebut juga dengan Fault Tolerant CAN. Karakteristik sinyal pada low speed CAN bus dapat dilihat seperti ini :

Recessive (high / 1) terjadi jika perbedaan tegangan pada terminal CAN High dan CAN Low kurang dari +0.6 volt
Dominant (low / 0) terjadi jika perbedaan tegangan pada terminal CAN High dan CAN Low antara +2.3 volt sampai +5 volt

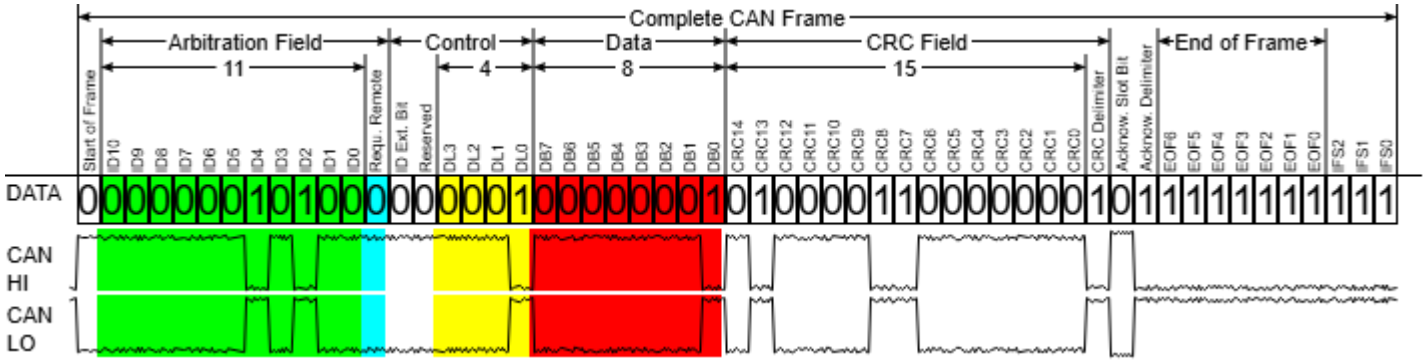


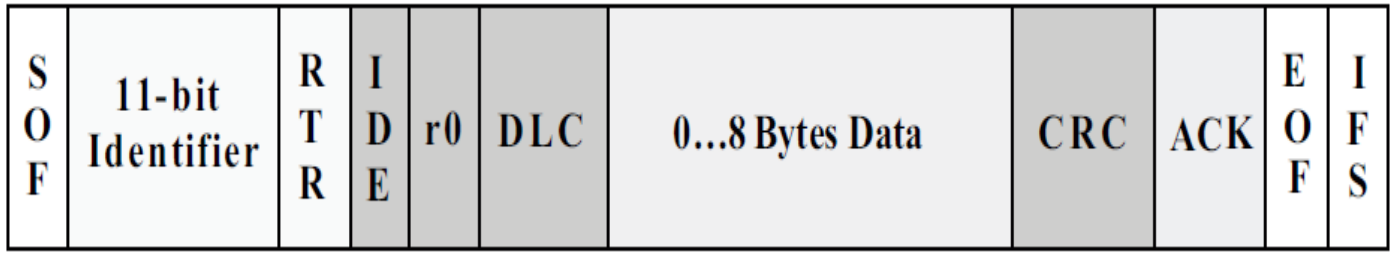
3.2 Frame Data CAN Bus

CAN Bus dikenal beberapa frame yang digunakan sebagai standar untuk melakukan transmisi data. Frame-frame tersebut adalah data frame, remote frame, error frame, overload frame.

3.2.1 Data Frame

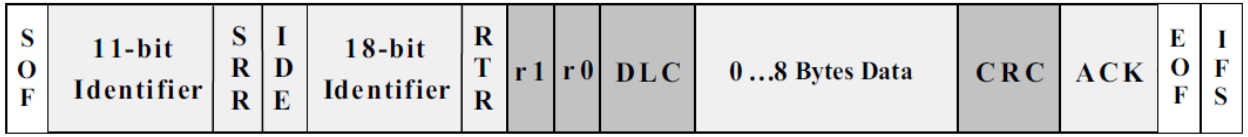
Data frame digunakan ketika sebuah node atau perangkat ingin mengirimkan data kepada node lainnya. Data frame berisi informasi tentang ID, data length dan data itu sendiri. ID merupakan identitas node pengirim data. Pada CAN Bus, ID menentukan prioritas dari data yang ditransmisikan. Jika ada lebih dari satu node dalam satu jaringan yang mengirimkan data, maka node dengan ID terendah (prioritas tertinggi) yang akan tetap mengirimkan data. Sementara node lain harus menunda pengiriman data sampai data yang ditransmisikan oleh node dengan ID terendah selesai. Data length berisi informasi tentang total bytes data dalam satu kali transmisi. Maksimal jumlah byte yang diizinkan dalam satu kalu transmisi adalah 8 byte atau 64 bit data. Sementara informasi data akan disertakan tergantung dari ukuran byte yang sudah dideskripsikan sebelumnya pada data length. Misalkan data length berisi 1 byte maka hanya ada 1 byte data atau 8 bit data yang akan ditransmisikan. Selain ID, data length dan data bit, ada bit-bit lain juga yang disertakan dalam pengiriman data. Perlu diketahui bahwa data frame pada standar CAN 2.0 A dan CAN 2.0 B sedikit berbeda. Namun perbedaannya tidak terlalu signifikan, perbedaan ini dikarenakan oleh ukuran bit ID yang berbeda antara CAN 2.0 A dan CAN 2.0 B.





Nama Signal	Ukuran (bit)	Fungsi	Nilai
Start Frame	1	Sinyal penanda akan dilakukan transmisi data	Dominant (Low)
ID (Identifier) (warna hijau)	11	ID unik dari masing-masing perangkat yang merepresentasikan prioritas dari data yang akan dikirim	Tergantung ID dari node
Remote transmission request (RTR)	1	Sinyal untuk mengirim data atau melakukan request data	Dominant (low) untuk data frame (node mengirimkan data)
Identifier extension bit (IDE)	1	Sinyal penanda menggunakan 11 bit ID (CAN 2.0 A) atau 29 bit ID (CAN 2.0 B)	Dominant (low) untuk 11 bit ID (CAN 2.0 B)
Reserved bit (r0)	1	Bit data cadangan, saat ini belum digunakan	Dominant (low)
Data length code (DLC) (warna kuning)	4	Panjang data yang akan ditransmisikan	Tergantung panjang data yang akan ditransmisikan (0-8)
Data field (warna merah)	0-64	Data yang akan ditransmisikan	Tergantung data yang akan dikirim
CRC	15	Sinyal yang digunakan untuk melakukan pengecekan error	Tergantung hasil perhitungan CRC
CRC delimiter	1	Sinyal pembatas CRC	Recessive (high)
ACK slot	1	Sinyal perrnyataan bahwa transmisi data telah selesai	Node transitter akan mengirimkan sinyal recessive. Sedangkan node receiver akan mengirimkan sinyal dominant
ACK delimiter	1	Sinyal pembatas ACK	Recessive (high)
End-of-frame (EOF)	7	Sinyal petanda bahwa transmisi data sudah complete	Recessive sebanyak 7 bit data

Data Frame CAN 2.0 B



Nama Signal	Ukuran (bit)	Fungsi	Nilai
Start Frame	1	Sinyal penanda akan dilakukan transmisi data	Dominant (Low)
ID (Identifier) (warna hijau)	11	ID unik dari masing-masing perangkat yang merepresentasikan prioritas dari data yang akan dikirim (11 bit pertama)	Tergantung ID dari node
Substitute remote request (SRR)	1		Recessive
Identifier extension bit (IDE)	1	Sinyal penanda menggunakan 11 bit ID (CAN 2.0 A) atau 29 bit ID (CAN 2.0 B)	Recessive untuk 29 bit ID (CAN 2.0 B)
ID (Identifier) (warna hijau)	18	ID unik dari masing-masing perangkat yang merepresentasikan prioritas dari data yang akan dikirim (18 bit kedua)	Tergantung ID dari node
Remote transmission request (RTR)	1	Sinyal untuk mengirim data atau melakukan request data	Dominant (low) untuk data frame (node mengirimkan data)
Reserved bit (r1, r0)	2	Bit data cadangan, saat ini belum digunakan	Dominant (low)
Data length code (DLC) (warna kuning)	4	Panjang data yang akan ditransmisikan	Tergantung panjang data yang akan ditransmisikan (0-8)
Data field (warna merah)	0-64	Data yang akan ditransmisikan	Tergantung data yang akan dikirim
CRC	15	Sinyal yang digunakan untuk melakukan pengecekan error	Tergantung hasil perhitungan CRC
CRC delimiter	1	Sinyal pembatas CRC	Recessive (high)
ACK slot	1	Sinyal perrnyataan bahwa transmisi data telah selesai	Node transitter akan mengirimkan sinyal recessive. Sedangkan node receiver akan mengirimkan sinyal dominant
ACK delimiter	1	Sinyal pembatas ACK	Recessive (high)
End-of-frame (EOF)	7	Sinyal petanda bahwa transmisi data sudah complete	Recessive sebanyak 7 bit data

3.2.2 Remote Frame

Remote frame merupakan frame yang digunakan untuk melakukan request data. Umumnya pada komunikasi CAN Bus tiap node akan melakukan pengiriman data tanpa adanya permintaan terlebih dahulu. Proses pengiriman data seperti ini wajib menggunakan data frame. Selain itu bisa juga terjadi proses pengiriman data atas permintaan node lain. Proses meminta data ini mewajibkan menggunakan format remote frame. Perbedaan remote frame dengan data frame adalah sebagai berikut :

Pada remote frame sinyal RTR selalu bernilai recessive. Pada data frame sinyal RTR selalu bernilai dominant
Pada remote frame tidak terdapat data bit. Pada data frame pasti terdapat data bit
Pada remote frame sinyal DLC berfungsi untuk menentukan ukuran data yang akan di request. Sedangkan pada data frame sinyal DLC menentukan ukuran data yang akan dikirim

3.2.3 Error Frame

Error frame merupakan sinyal yang dikirimkan jika terjadi error atau kesalahan pada proses transmisi data. Error frame terdiri dari sinyal error flag dan error delimiter. Error flag merupakan sinyal error itu sendiri, yang terdiri dari 6-12 bit sinyal recessive atau dominant. Sedangkan error delimiter merupakan sinyal yang dikirimkan setelah error flag, yang berfungsi sebagai petanda bahwa error flag telah dikirim. Sinyal error delimiter terdiri dari 8 bit sinyal dengan nilai recessive (high). Error frame akan dikirimkan oleh node jika node tersebut mendeteksi error pada CAN bus.

3.2.4 Overload Frame

Overload frame merupakan sinyal yang akan dikirimkan sebuah node jika node tersebut tidak mampu menampung data yang dikirimkan oleh node lain. Hal ini dapat terjadi karena terlalu cepatnya proses transfer data, sehingga node receiver tidak mampu mengolah data yang diterima atau disebut dengan overload. Ketika terjadi overload, node receiver dapat mengirimkan sinyal overload frame yang berfungsi untuk menginformasikan ke node transmitter agar proses pengiriman data diberi jeda waktu (delay).

3.4 Penanganan Error Pada CAN Bus Error

Salah satu kelebihan CAN Bus adalah pada kemampuannya untuk menangani error yang terjadi pada komunikasi data. Proses penanganan error pada CAN Bus melibatkan beberapa proses di bawah ini.

3.4.1 Arbitration

Abritation merupakan salah satu fitur penanganan error yang ada pada CAN Bus. Abritation berfungsi untuk menangani jika terjadi pengiriman data yang berbarengan antara dua atau lebih node. Pada komunikasi data multi point menggunakan EIA-485, tidak diijinkan ada dua perangkat yang mengirimkan data secara bersamaan dalam satu jaringan yang sama. Hal ini akan menyebabkan error pada data yang dikirimkan. Namun, hal ini tidak terjadi pada CAN Bus. Semua node boleh saja mengirimkan data secara bersamaan, namun pada akhirnya node yang memiliki ID lebih rendah (prioritas lebih tinggi) yang akan tetap mengirimkan data. Node yang lainnya akan mengalah dengan cara menunggu sampai proses pengiriman data dari node yang memiliki prioritas lebih tinggi selesai dan kemudian mengirimkan data ulang. Proses ini dikenal dengan nama arbitration.

	Start Bit	ID Bits											The Rest of the Frame
		10	9	8	7	6	5	4	3	2	1	0	
Node 15	0	0	0	0	0	0	0	0	1	1	1	1	
Node 16	0	0	0	0	0	0	0	1	Stopped Transmitting				
CAN Data	0	0	0	0	0	0	0	0	1	1	1	1	

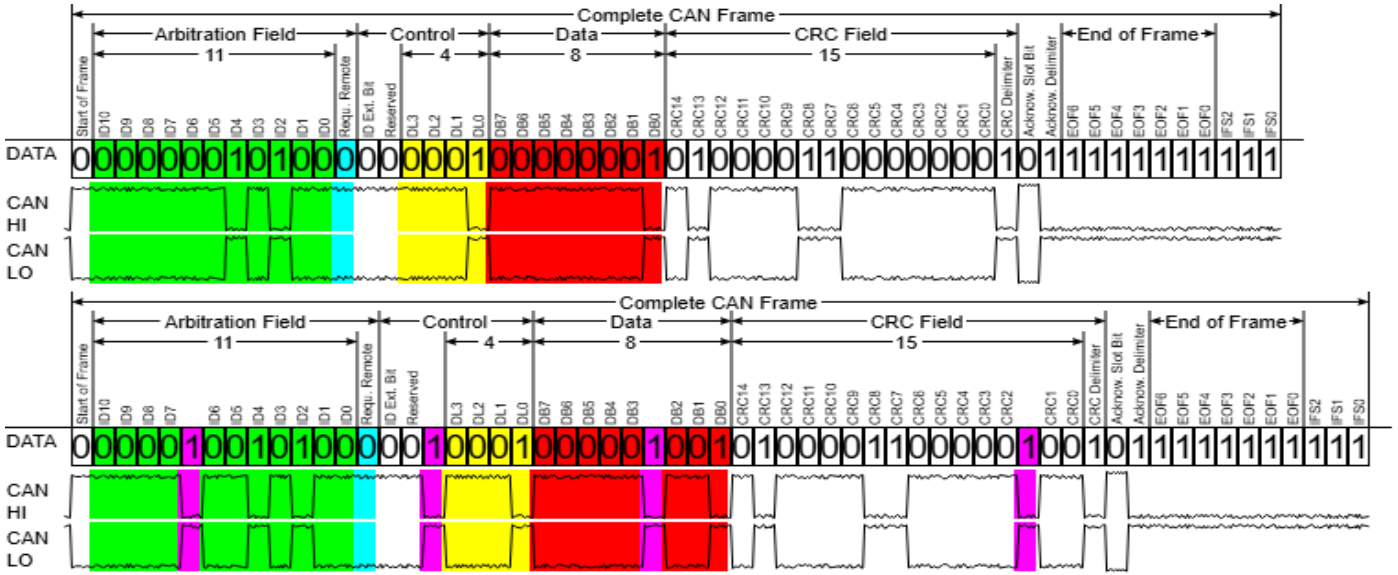
Proses arbitration lebih jelasnya dapat dilihat pada gambar di atas. Gambar diatas menjelaskan bahwa ada dua node yang sama-sama mengirimkan data. Node tersebut memiliki ID 15 dan ID 16. Seperti yang telah dijelaskan pada materi sebelumnya, proses pengiriman data akan menggunakan data frame. Sinyal start akan dikirimkan terlebih dahulu untuk memberikan petanda bahwa akan terjadi proses pengiriman data. Setelah sinyal start dikirim, sinyal selanjutnya yang dikirimkan adalah sinyal ID (Identifier) dari node yang mengirimkan data. Pada contoh diatas standar CAN yang digunakan adalah CAN 2.0 A dimana ID terdiri dari 11 bit ID. Node dengan ID 15 akan mengirimkan sinyal 0b00000001111, sedangkan node dengan ID 16 akan mengirimkan sinyal 0b00000010000. Pada saat mentransmisikan sinyal ID pada bit ke 4 akan terjadi perbedaan antara node 15 dan node 16. Dimana node 15 sinyalnya adalah dominat (0) sementara node 16 sinyalnya adalah recessive (1). Pada saat seperti ini, sinyal dominant akan menjadi prioritas. Sehingga membuat node 16 menghentikan transmisi data sementara dan menunggu sampai transmisi data dari node 15 selesai. Proses arbitration ini menjadi jaminan tidak ada kesalahan pengiriman data walaupun ada lebih dari dua node yang mengirimkan data secara bersamaan.

3.4.2 Synchronization

CAN Bus merupakan jenis komunikasi data asinkron. Dimana sinyal data akan dikirimkan tanpa sinyal clock. Proses transmisi data akan dilakukan dengan kesepakatan kecepatan transfer data yang sama pada semua node. Sehingga sebuah node dapat mengirim dan menerima data dengan timing yang sama tanpa perlu membandingkan sengan sinyal clock. Namun, pada kenyataannya terdapat sedikit perbedaan kecepatan transfer data antar node. Hal ini bisa disebabkan oleh toleransi dari beberapa komponen yang berfungsi sebagai clock generator pada masing-masing node. Perbedaan kecepatan transfer data dari masing-masing node dapat menyebabkan kesalahan pada saat mengirim ataupun menerima data. CAN Bus menggunakan proses sinkronisasi untuk menangani kesalahan ini. Proses sinkronisasi dilakukan pada saat terjadi perubahan sinyal dari recessive ke dominant. Proses sinkronisasi pertama dilakukan pada saat perubahan antara kondisi idle ke sinyal start. Dimana pada kondisi idle sinyal pada CAN Bus adalah recessive kemudian saat sinyal start ditransmisikan sinyal pada CAN Bus berubah menjadi dominant. Pada saat terjadi transisi inilah node akan mulai melakukan sinkronisasi dengan cara menghitung jumlah bit data dan lama waktu sampai terjadi transisi berikutnya. Sehingga diperoleh lama nilai t (periode) dari 1 bit data.

3.4.3 Bit Stuffing

Bit stuffing merupakan proses menambahakan beberapa bit data pada sebuah frame. Tujuan dari bit stuffing ini adalah untuk membantu proses sinkronisasi yang telah dijelaskan sebelumnya. Bit stuffing digunakan untuk memastikan transisi pada frame data cukup untuk melakukan sinkronisasi.



Pada gambar di atas dapat dilihat bahwa normalnya frame data akan dikirimkan seperti gambar pertama. Dimana ID dari node yang mengirim adalah 20. Panjang data yang akan dikirim adalah 1 byte. Sementara data yang dikirim adalah 1 diikuti dengan sinyal lainnya. Penambahan bit pada frame diatas akan dilakukan jika terdapat 5 bit yang memiliki nilai sama dan berurutan. Satu bit akan ditambahkan setelah kelima bit yang sama dan berurutan tersebut. Nilai bit nya merupakan kebalikan dari nilai lima bit yang berurutan tersebut. Pada gambar diatas perhatikan bit yang berwarna ungu. Bit yang berwarna ungu merupakan contoh dari bit stuffing

3.4.4 ACK Signal

Sinyal ACK merupakan sinyal penanda bahwa tidak ada error pada saat pengiriman data. Sinyal ini akan dikirimkan oleh node transmitter dan receiver. Transmitter akan mengirimkan sinyal recessive pada bit ACK. Node receiver akan mengecek data yang diterima. Jika receiver tidak mendeteksi adanya error pada data maka node receiver akan mengirimkan sinyal dominant. Dimana sinyal dominant akan menggantikan sinyal recessive yang sebelumnya dikirim oleh transmitter. Ketika sinyal ACK bernilai dominant (0) ini berarti bahwa tidak ada kesalahan atau error pada saat transmisi data. Namun jika node receiver mendeteksi error maka akan mengirimkan sinyal recessive pada bit ACK. Hal ini menunjukkan bahwa ada kesalahan pada saat transmisi data. Jika dalam satu jaringan terdapat banyak node, semua node akan merespon dengan memberikan sinyal ACK tergantung data yang diterima pada masing-masing node error atau tidak. Jika ternyata hanya ada satu node yang mendeteksi error dan node lainnya tidak, maka transmitter akan menganggap tidak ada error. Transmitter akan mendeteksi error jika semua node memberikan sinyal recessive.

3.4.5 Interframe Spacing

Interframe spacing merupakan jarak pengiriman data antar frame. Interframe spacing berfungsi untuk memberikan jeda waktu antar pengiriman data. Interframe spacing membutuhkan minimal tiga bit recessive sebagai jarak antar frame. Adanya interframe spacing dapat memberikan waktu bagi processor untuk menyimpan data yang sebelumnya dikirim sebelum adanya transmisi data yang baru.

3.4.6 Cyclic Redundancy Check

CRC (Cyclic Redundancy Check) merupakan 15 bit sinyal yang digunakan untuk melakukan pengecekan error terhadap data yang dikirimkan. CRC tidak hanya digunakan pada CAN Bus tetapi digunakan juga pada protokol komunikasi data lainnya contohnya Modbus. Bit CRC merupakan hasil perhitungan algoritma CRC dari data yang dikirimkan. Pada saat data diterima, data akan dibandingkan dengan nilai CRC-nya. Jika data dan nilai CRC-nya sesuai berarti tidak terjadi error pada saat pengiriman data. Namun, jika data dan nilai CRC-nya berbeda, maka bisa dipastikan data yang dikirimkan error.

4. Ethernet

Ethernet pada awalnya dikembangkan oleh perusahaan Xerox pada awal tahun 1970. Untuk saat ini, etehrnet telah menjadi standar komunikasi yang diakui IEEE. Standar untuk ethernet sendiri dikenal dengan nama IEEE 802.3. Ethernet dapat digunakan untuk komunikasi data point to pint maupun multipoint. Komunikasi data ethernet memiliki kecepatan 10Mbps. Namun, untuk saat ini pengembangan ethernet membuat kemampuan transmisi datanya semakin cepat bisa mencapai 100Mpbs. Ethernet dengan kecepatan 100Mbps biasanya dikenal dengan nama fast ethernet. Jarak jangkauan maksimal untuk komunikasi data ethernet adalah 500 meter. Namun jangkauan bisa diperluas dengan menggunakan repeater atupun bridge. Pada komunikasi ethernet semua perangkat yang terkoneksi pada satu jaringan dapat melakukan transmisi data satu dan lainnya sama seperti pada CAN Bus.

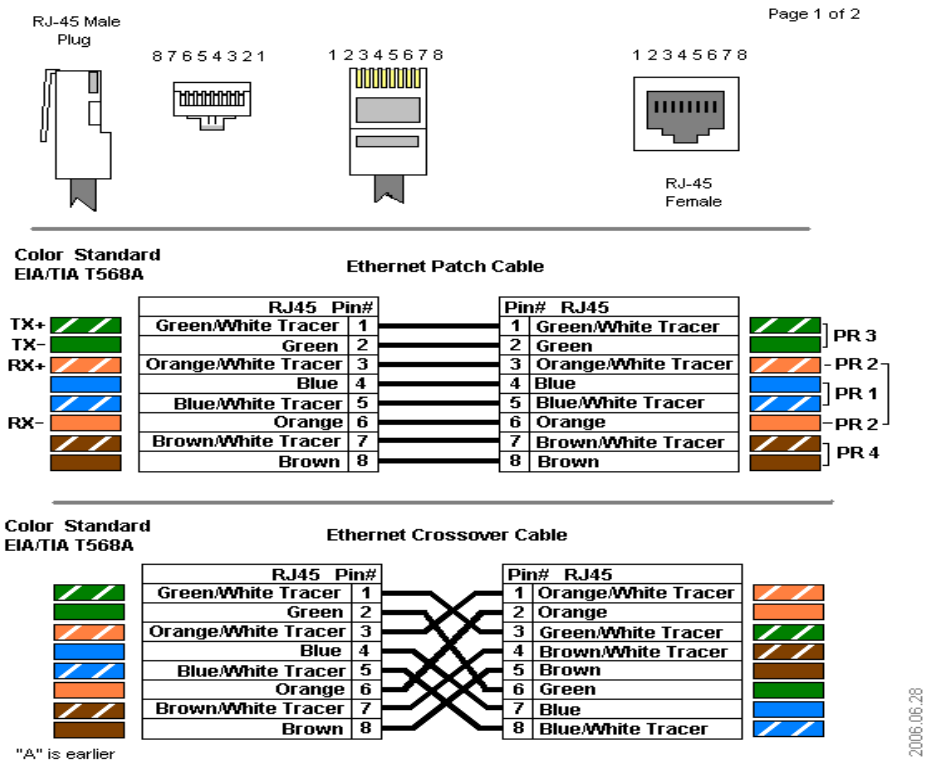
4.1 IP Address (Internet Protocol Address)

IP Address merupakan salah satu identifikasi yang diberikan pada sebuah perangkat yang terkoneksi pada jaringan ethernet. Masing-masing perangkat yang terkoneksi ke jaringan ethernet harus memiliki IP Address yang unik satu dengan yang lainnya. IP Address memiliki ukuran 32 bit. IP Address dibagi menjadi dua bagian yaitu network ID dan host ID. Jumlah maksimal perangkat yang dapat terkoneksi ke jaringan ethernet dapat ditentukan oleh Class dari IP Address yang digunakan. Class pada IP Address dapat dilihat di bawah ini

a) Class A, memiliki ukuran host ID 24bit. Sehingga maksimal perangkat yang dapat terkoneksi adalah 16.777.216 perangkat dalam satu jaringan
b) Class B, memiliki ukuran host ID 16bit. Sehingga maksimal perangkat yang dapat terkoneksi adalah 65536 perangkat dalam satu jaringan
c) Class C, memiliki ukuran host ID 8bit. Sehingga maksimal perangkat yang dapat terkoneksi adalah 256 perangkat dalam satu jaringan
d) Class D digunakan khusus untuk mkomunikasi data multicast
e) Class E digunakan untuk pengembangan ke depan

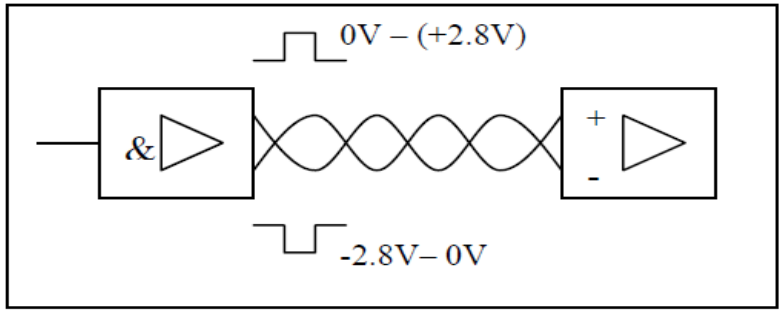
4.2 Karakteristik Sinyal Ethernet

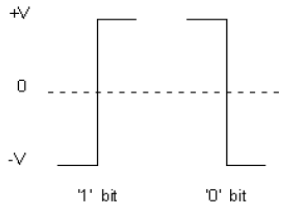
Komunikasi ethernet, data ditransmisikan dalam bentuk differential wire sama seperti EIA-485 dan CAN Bus. Pada ethernet masing-masing perangkat memiliki jalur transmitter dan jalur receiver tersendiri. Jalur transmitter memiliki terminal TX+ dan TX-. Sedangkan jalur receiver memiliki terminal RX+ dan RX-.

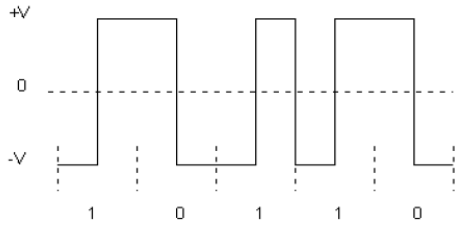


Sinyal pada komunikasi ethernet dengan standar IEEE 802.3 menggunakan pengkodean sinyal manchester. Dimana :

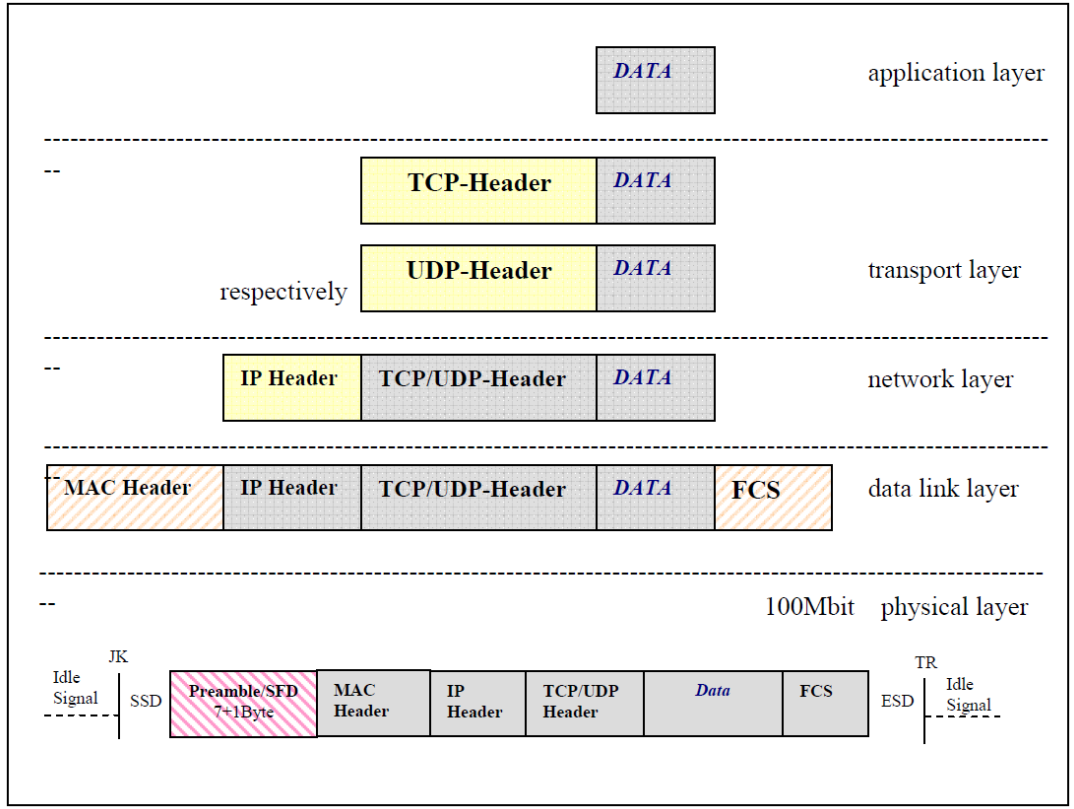
- Sinyal high (1), merupakan transisi dari 0 ke 1
- Sinyal low (0), merupakan transisi dari 1 ke 0



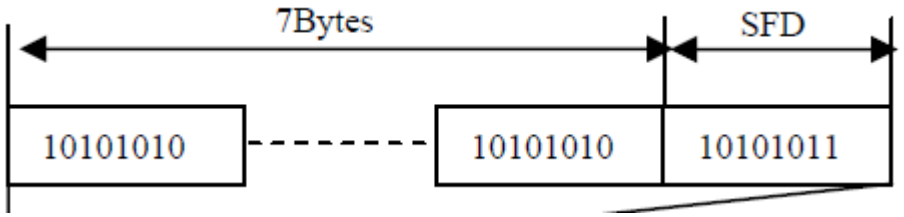




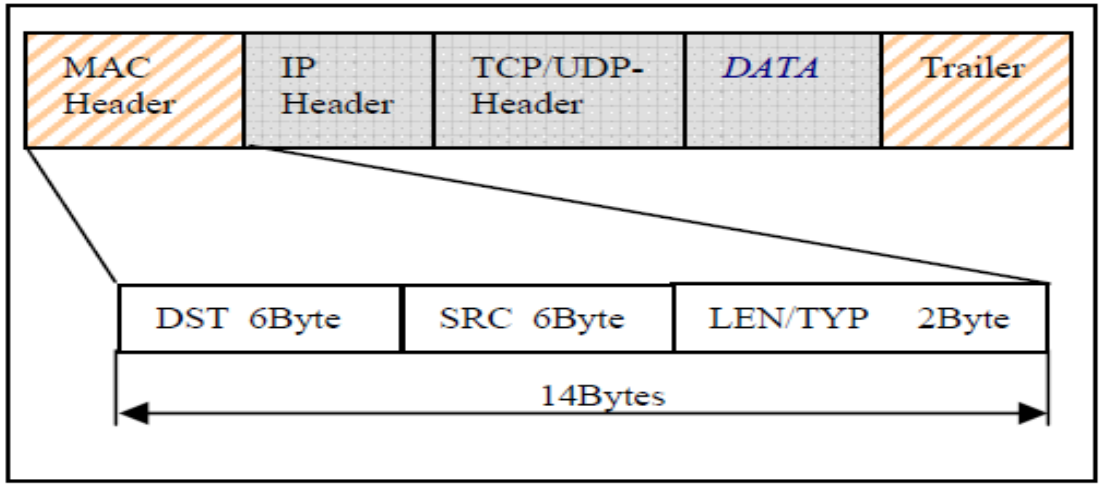
4.2 Frame Data Ethernet



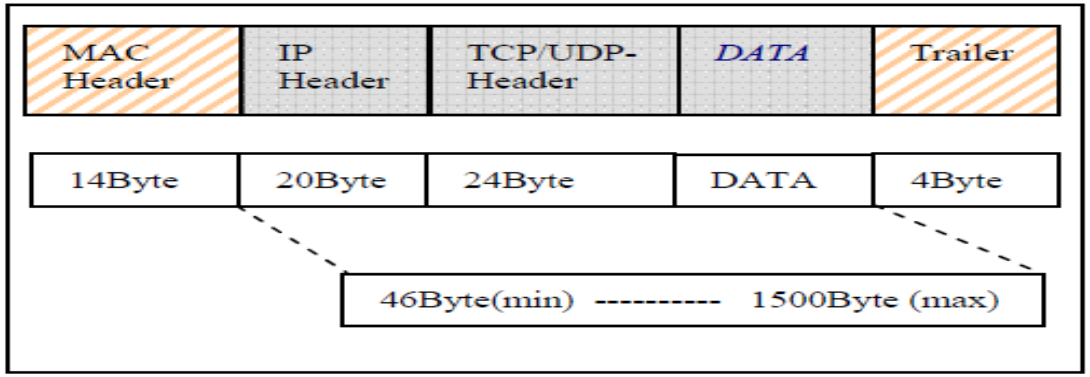
1. SSD, Start of Frame merupakan sinyal penanda awal pengiriman data.
2. Preamble, merupakan bit yang digunakan untuk melakukan sinkronisasi. Bit preamble terdiri dari 7 byte sinyal sinkronisasi dan ditambahkan 1 byte sinyal SFD yang merupakan sinyal delimiter dari preambble.

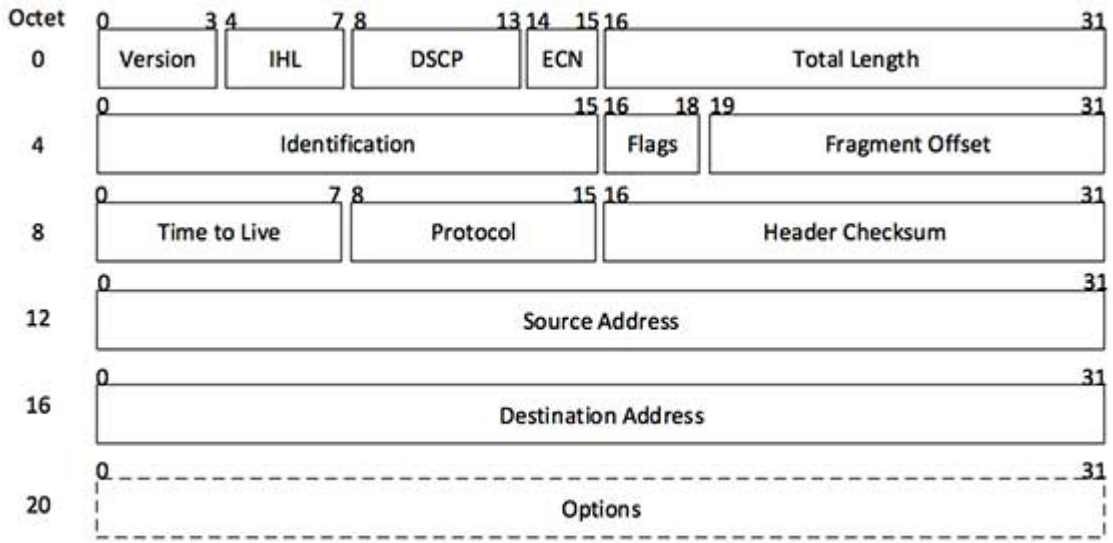


3. MAC Header, berisi informasi tentang MAC Address pengirim (6 byte) dan MAC Addreess penerima (6 byte) serta 2 byte mencakup informasi panjang data yang akan dikirimkan atau tipe service yang digunakan pada saat pengiriman data. Pada standar IEEE802.3 2 byte terakhir berisi informasi panjang data yang akan dikirim (0-1500).



4. IP Header, berisi informasi IP Address pengirim dan IP address penerima. TCP/UDP Header, protokol data yang digunakan pada pengiriman data. Data, merupakan sinyal data yang akan dikirimkan. Ukuran panjang data (IP Header + TCP/UDP Header + DATA) yang dapat dikirimkan dalam satu paket data minimal adalah 46 byte dan maksimal adalah 1500byte dalam sekali pengiriman frame data





[Image: IP Header]

Version: Version no. of Internet Protocol used (e.g. IPv4).

IHL: Internet Header Length; Length of entire IP header.
DSCP: Differentiated Services Code Point; this is Type of Service.
ECN: Explicit Congestion Notification; It carries information about the congestion seen in the route.
Total Length: Length of entire IP Packet (including IP header and IP Payload).
Identification: If IP packet is fragmented during the transmission, all the fragments contain same identification number. to identify original IP packet they belong to.
Flags: As required by the network resources, if IP Packet is too large to handle, these 'flags' tells if they can be fragmented or not. In this 3-bit flag, the MSB is always set to '0'.
Fragment Offset: This offset tells the exact position of the fragment in the original IP Packet.
Time to Live: To avoid looping in the network, every packet is sent with some TTL value set, which tells the network how many routers (hops) this packet can cross. At each hop, its value is decremented by one and when the value reaches zero, the packet is discarded.
Protocol: Tells the Network layer at the destination host, to which Protocol this packet belongs to, i.e. the next level Protocol. For example protocol number of ICMP is 1, TCP is 6 and UDP is 17.
Header Checksum: This field is used to keep checksum value of entire header which is then used to check if the packet is received error-free.
Source Address: 32-bit address of the Sender (or source) of the packet.
Destination Address: 32-bit address of the Receiver (or destination) of the packet.
Options: This is optional field, which is used if the value of IHL is greater than 5. These options may contain values for options such as Security, Record Route, Time Stamp, etc.

TCP Segment Header Format

Bit #	0 – 7	8 – 15	16 – 23	24 – 31
0	Source Port		Destination Port	
32	Sequence Number			
64	Acknowledgment Number			
96	Data Offset	Res	Flags	Window Size
128	Header and Data Checksum		Urgent Pointer	
160...	Options			

UDP Datagram Header Format

Bit #	0 – 7	8 – 15	16 – 23	24 – 31
0	Source Port		Destination Port	
32	Length		Header and Data Checksum	

TCP Segment Header Format

Bit #	0 – 7	8 – 15	16 – 23	24 – 31
0	Source Port		Destination Port	
32	Sequence Number			
64	Acknowledgment Number			
96	Data Offset	Res	Flags	Window Size
128	Header and Data Checksum		Urgent Pointer	
160...	Options			

UDP Datagram Header Format

Bit #	0 – 7	8 – 15	16 – 23	24 – 31
0	Source Port		Destination Port	
32	Length		Header and Data Checksum	

5. FCS, merupakan sinyal yang digunakan untuk pengecekan error. FCS sama halnya dengan CRC pada CAN Bus. Sinyal FCS pada ethernet memiliki panjang 4 bytes.

Berdasarkan packet data diatas dalam sekali penmgiriman data menggunakan ethernet jumlah byte minimal dan maksimal yang dikirim adalah sebagai berikut.

Field Name	min Size	max Size	min w/o Preamble	max w/o Preamble
Preamble	7	7		
SFD	1	1		
DST	6	6	6	6
SRC	6	6	6	6
Type/Length	2	2	2	2
Data	46	1500	46	1500
Checksum	4	4	4	4
Total	72	1526	64	1518

4.3 Mode Operasi Ethernet

Pada komunikasi ethernet dapat digunakan mode komunikasi simplex, half-duplex ataupun full duplex. Pada ethernet jalur pengirim dan penerima dipisah dan dapat digunakan masing-masing. Sehingga memungkinkan untuk melakukan pengiriman dan penerimaan data secara bersamaan sekaligus.

4.4 Protokol Komunikasi Data Ethernet

Pada komunikasi data ethernet, terdapat dua protokol komunikasi data yang dapat digunakan yaitu TCP dan UDP. Penjelasan lebih detail tentang kedua protokol tersebut dapat dilihiat di bawah ini

Port dan Socket

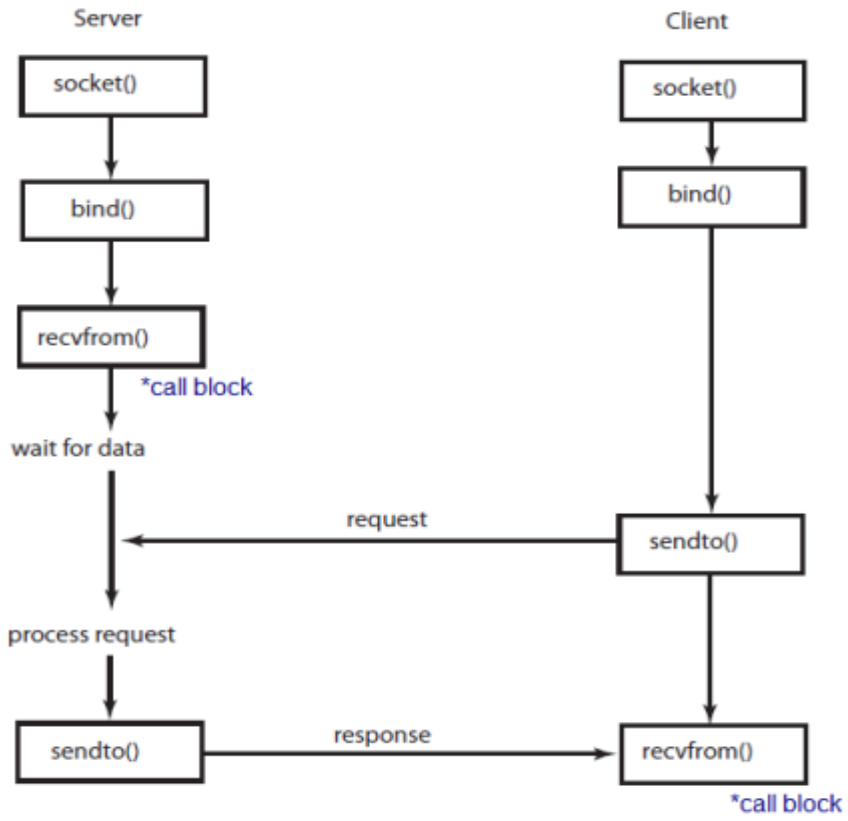
Pada komunikasi data ethernet terdapat istilah port dan socket. Port merupakan jalur keluar dan masuknya data. Port pada komunikasi data ethernet didefinisakan pada TCP header atau UDP header. Port disini bukan berarti konektor untuk menghunbungkan kabel. Port pada komunikasi ethernet merupakan nomor unik yang menentukan standar protokol yang digunakan. Nomer port terdiri dari 16bit nomer unik. Berarti terdapat maksimal 65536 nomor port.

Beberapa nomor port sudah digunakan untuk protokol komunikasi standar seperti port 80 untuk protokol HTTP, port 23 untuk Telnet, dll. Namun ada beberapa nomer port yang masih belum digunakan untuk protokol tertenttu. Port yang belum digunakan ini bisa kita gunakan untuk melakukan komunikasi data tanpa protokol tertentu.

Socket merupakan istilah yang digunakan untuk menunjukkan IP Address dan port yang digunakan untuk komunikasi data. Pada komunikasi ethernet data akan dikirimkan secara spesifik ke tujuan tertentu (IP Address) dan ke nomer port tertentu (Port). Serta dikirimkan dengan infromasi IP Address pengirim dan port pengirim.

4.4.1 UDP

UDP merupakan salah satu protokol komunikasi data pada komunikasi ethernet. Komunikasi data menggunakan protokol UDP tidak menjamin data sampai pada tujuan atau yang dikenal dengan nama Connectionless Oriented. Data akan dikirimkan secara broadcast ke penerima dengan IP address dan port terterntu.

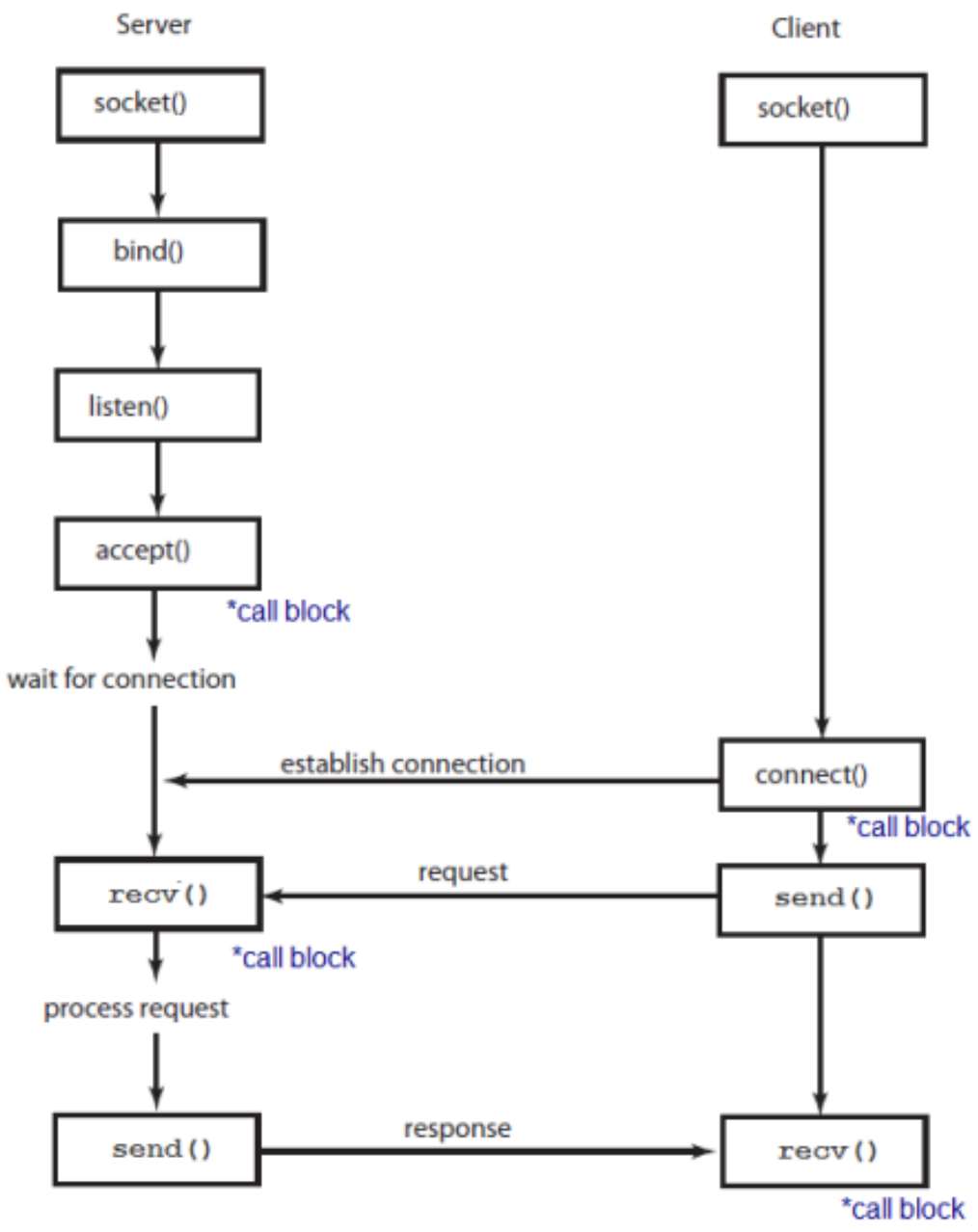


Gambar diatas menjelaskan proses komunikasi data menggunakan UDP. Server dan client sama-sama memulai komunikasi data dengan cara membuka akses pada nomor port yang telah ditentukan. Setelah itu data dikirimkan dari client ke server atau server ke client. Secara normal proses komunikasi akan berjalan lancar. Namun jika pada saat komunikasi data tiba-tiba koneksi terputus, baik client ataupun server tidak dapat memastikan bahwa data yang dikirim ternyata tidak sampai pada tujuan. Hal ini dikarenakan protokol UDP tidak menggunakan proses acknowledgement.

4.4.2 TCP

TCP merupakan protokol komunikasi data pada ethernet yang dapat menjamin bahwa data yang dikirimkan sampai ke tujuan. Hal ini disebabkan karena pada TCP kedua perangkat yang saling berkomunikasi harus melakukan proses persetujuan terlebih dahulu (acknowledgement). Kedua perangkat harus saling sepakat terlebih dahulu untuk dapat bertukar data. Jika pada pengiriman data ternyata terdapat error atau data tidak sampai ke tujuan, data akan dikirimkan ulang oleh pengirim.

Proses komunikasi data pada TCP dimulai dengan membuka akses ke port yang digunakan. Setelah itu dari sisi client melakukan request untuk membuka koneksi ke server. Proses ini memerlukan persetujuan server agar komunikasi data dapat terjadi. Setelah proses persetujuan terjadi kedua perangkat dapat saling bertukar data.



4.5 Percobaan Transmisi Data Melalui Ethernet

4.5.1 Percobaan Mengirim Data dengan UDP

Pada percobaan kali ini akan diuji coba pengiriman data antara dua perangkat melalui komunikasi ethernet menggunakan protokol UDP. Uji coba komunikasi data dilakukan pada Arduino dengan tambahan ethernet shield dan ethernet port pada komputer. Komputer akan membuka akses port 8888 untuk menerima data dari Arduino. IP Address yang digunakan oleh Arduino adalah "192.168.1.177". Sedangkan IP Address yang digunakan pada komputer adalah "192.168.1.2".

PING request?

ICMP

How to calculate Frame size for PING request?

In a Standard ethernet TCP/IP, Segment size is 1480 bytes. And the packet size is 1480 + 20 bytes (IP) = 1500 bytes. In the frame, 1500 bytes is MTU size with frame header 26 bytes (Preamble + SOF + Dest. MAC + ORIG. MAC + CRC).

Now, on internet, I found MTU size of ICMP frame is 1500 - 20 bytes (IP header) - 8 bytes (ICMP header) = 1472 bytes. I failed to understand this calculation if the ethernet frame MTU is 1500 bytes, then why are we sending 1472 bytes for ICMP frames ?

Or to put simply, what 1472 bytes of ICMP MTU consists of ?

1500 bytes is MTU size without Ethernet frame header -- it's the transmission size of Ethernet data, not the whole Ethernet frame. More specifically, it's the whole size of IP packet.
Similarly, the MTU of ICMP is the transmission size of ICMP data, not the whole ICMP message. Thus, the number would be 1500 (max data size of Ethernet) - 20 (IP header) - 8 (ICMP header) = 1472

1. Format Frame Data (Enkapsulasi)

Data ICMP tidak berjalan sendirian; ia dibungkus (dienkapsulasi) dalam beberapa lapisan protokol.

- Ethernet Frame Header (Layer 2): 14 byte (Alamat MAC Sumber, Tujuan, dan EtherType).
- IP Header (Layer 3): 20 byte (berisi alamat IP sumber dan tujuan).
- ICMP Header (Layer 4/3): 8 byte (berisi Type, Code, Checksum, Identifier, dan Sequence Number).
- Payload/Data: Ukuran variabel (berisi data yang akan dikirim).
- FCS (Frame Check Sequence): 4 byte (opsional, sering tidak terlihat di alat seperti Wireshark).

Since the CRC bits are transmitted as part of the frame we should count them in the byte count of the frame:
6 bytes DA + 6 bytes SA + 2 bytes Type + 20 bytes IP header + 4 bytes ICMP header + 4 bytes identifier+sequence number + 32 bytes actual data = 74
Here is how 74 is calculated.

Since the CRC bits are transmitted as part of the frame we should count them in the byte count of the frame:
6 bytes DA + 6 bytes SA + 2 bytes Type + 20 bytes IP header + 4 bytes ICMP header + 4 bytes identifier+sequence number + 32 bytes actual data = 74

Here is how 74 is calculated.

how it became 74:
ip header = 20, ethernet frame minimum is 64 bytes if not specify the size.
Inside ethernet frame = icmp header (8bytes) + data.
Data is 64 bytes - 8 bytes = 56 bytes.
So overall:
ip header (20) + icmp header (8) + data (56) = 74 bytes.
example:
ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: icmp_seq=0 ttl=57 time=11.643 ms
64 bytes from 8.8.8.8: icmp_seq=1 ttl=57 time=20.087 ms

2. Kapasitas Maksimum Pengiriman Data ICMP

Kapasitas maksimum data yang dapat dikirim dalam satu paket ICMP ditentukan oleh MTU (Maximum Transmission Unit) dari jaringan Ethernet standar, yaitu 1.500 byte.
Jika kita menghitung mundur dari MTU 1.500 byte:

a) Total MTU: 1.500 byte.
b) Dikurangi IP Header: -20 byte.
c) Dikurangi ICMP Header: -8 byte.
d) Maksimum Payload Data: 1.472 byte

Komponen	**Ukuran (Byte)**	**Akumulasi**
Payload Data	1.472	1.472
ICMP Header	8	1.480
IP Header	20	1.500 (MTU)
Ethernet Header	14	1.514

3. **Mengapa 1.472 Byte Sangat Penting?**

Jika Anda mengirim data ICMP lebih dari 1.472 byte (misalnya ping -l 1500), paket tersebut akan mengalami fragmentasi. Fragmentasi memecah paket besar menjadi beberapa paket kecil agar sesuai dengan MTU jaringan, namun ini dapat meningkatkan beban kerja router dan risiko paket hilang.
Contoh Perintah Pengiriman Maksimal:
Pada Windows, Anda bisa mencoba mengirim data tepat pada batas MTU tanpa fragmentasi dengan perintah:
ping [alamat_ip] -l 1472 -f

- -l 1472: Mengatur ukuran payload data sebesar 1.472 byte.
- -f: Mengatur Don't Fragment flag, sehingga jika paket terlalu besar, jaringan akan menolaknya daripada memecahnya.

Format frame data dan ukuran maksimum untuk data yang dikirim melalui ICMP (khususnya Echo Request/Ping), berdasarkan struktur standar IPv4 dan Ethernet

1. Format Frame Data (Ethernet + IP + ICMP)

ICMP dikemas (*encapsulated*) di dalam paket IP, yang kemudian dimasukkan ke dalam frame Ethernet.

Lapisan	Komponen	Ukuran (Bytes)	Deskripsi
Layer 2	Ethernet Header	14	MAC Address Tujuan, Sumber, Type
Layer 3	IP Header	20 - 60	IPv4 (Umumnya 20 bytes)
Layer 3	ICMP Header	8	Type, Code, Checksum, ID, Sequence
Layer 3	ICMP Payload/Data	Variabel	Data yang dikirim (contoh: ABCDE...)
Layer 2	Ethernet Trailer	4	FCS (Frame Check Sequence)

Total Frame Data = Ethernet Header + IP Header + ICMP Header + ICMP Payload + FCS.

2. ICMP Send Data Max (Ukuran Maksimum)

Ukuran data maksimum yang dapat dikirim oleh ICMP ping tanpa fragmentasi IP bergantung pada MTU (Maximum Transmission Unit) jaringan, yang biasanya bernilai 1500 bytes pada Ethernet.

- MTU: 1500 bytes
- IP Header: 20 bytes (standar)
- ICMP Header: 8 bytes
- Maksimum ICMP Payload (Data): 1500-20-8 = 1472 bytes

Jika Anda mengirim data lebih dari 1472 bytes (ditambah 28 bytes header), paket akan terfragmentasi, yang dapat menyebabkan kehilangan paket atau pemblokiran oleh firewall.

3. ICMP Send Data Max (Ukuran Maksimum)

```
+-----------------------+-------------------+-----------------------+-----------------------------+
| Ethernet Header  |     IP Header    |   ICMP Header    |   ICMP Payload (Data) |
|    (14 bytes)          |    (20 bytes)     |    (8 bytes)          |   (Max 1472 bytes)      |
+-----------------------+-------------------+-----------------------+-----------------------------+
```

- Total Paket IP Maks (tanpa frag): 1500 bytes
- ICMP Header + Data Max: 1480 bytes (8+1472)
- ICMP Data Payload Max: 1472 bytes
- Default Ping (Windows): 32 bytes (Data) + 8 (Header) = 40 bytes

2. ICMP Send Data Max (Ukuran Maksimum)

Ukuran data maksimum yang dapat dikirim oleh ICMP ping tanpa fragmentasi IP bergantung pada MTU (Maximum Transmission Unit) jaringan, yang biasanya bernilai 1500 bytes pada Ethernet.

- MTU: 1500 bytes
- IP Header: 20 bytes (standar)
- ICMP Header: 8 bytes
- Maksimum ICMP Payload (Data): 1500-20-8 = 1472 bytes

Jika Anda mengirim data lebih dari 1472 bytes (ditambah 28 bytes header), paket akan terfragmentasi, yang dapat menyebabkan kehilangan paket atau pemblokiran oleh firewall.

3. ICMP Send Data Max (Ukuran Maksimum)

```
+-----------------------+-------------------+-----------------------+-----------------------------+
| Ethernet Header  |     IP Header    |   ICMP Header    |   ICMP Payload (Data) |
|    (14 bytes)          |    (20 bytes)     |    (8 bytes)          |   (Max 1472 bytes)      |
+-----------------------+-------------------+-----------------------+-----------------------------+
```

- Total Paket IP Maks (tanpa frag): 1500 bytes
- ICMP Header + Data Max: 1480 bytes (8+1472)
- ICMP Data Payload Max: 1472 bytes
- Default Ping (Windows): 32 bytes (Data) + 8 (Header) = 40 bytes